

Digital Design & Computer Arch.

Lecture 4a: Sequential Logic Design II Finite State Machines

Prof. Onur Mutlu

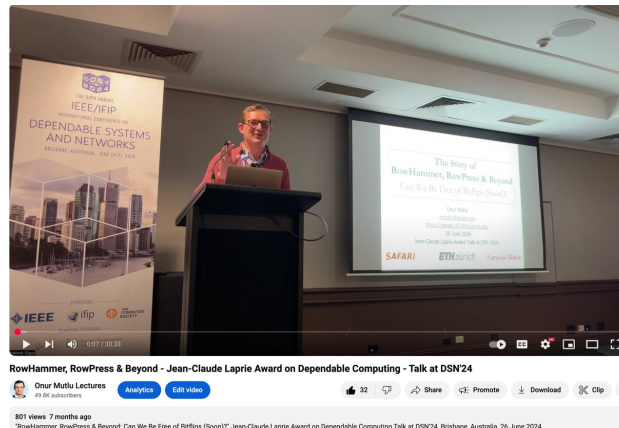
ETH Zürich

Spring 2026

27 February 2026

Extra Credit Assignment: Talk Analysis

- The Story of RowHammer, RowPress & Beyond
- Watch and analyze this short lecture (30 minutes)
 - ❑ <https://www.youtube.com/watch?v=U1EcqXlclKU> (June 2024)



- Assignment – for 1% extra credit
 - ❑ Write a good 1-page individualized summary (no AI use)
 - What are your key takeaways? What did you learn?
 - What surprised you about the content presented? What excited you?
 - What do you think solutions should be like?
 - Submit your summary to Moodle – deadline March 21

Agenda for Today & Next Week

■ Today

- Finish Sequential Logic
- Introduction to Labs
- Start Hardware Description Languages and Verilog

■ Next week

- Finish Hardware Description Languages and Verilog
 - Combinational Logic
 - Sequential Logic
- Timing and Verification

Readings

- Combinational Logic
 - P&P Chapter 3 until 3.3 + H&H Chapter 2
- Sequential Logic
 - P&P Chapter 3.4 until end + H&H Chapter 3 in full
- Hardware Description Languages and Verilog
 - H&H Chapter 4 in full
- Timing and Verification
 - H&H Chapters 2.9 and 3.5 + (start Chapter 5)

- By the end of next week, make sure you are done with
 - **P&P Chapters 1-3 + H&H Chapters 1-4**

First, We Will Complete Sequential Logic Design

We Covered How to Store Information

■ **Circuits that can store information**

- ❑ Cross-coupled inverter
- ❑ R-S Latch
- ❑ Gated D Latch
- ❑ D Flip-Flop
- ❑ Register
- ❑ Memory

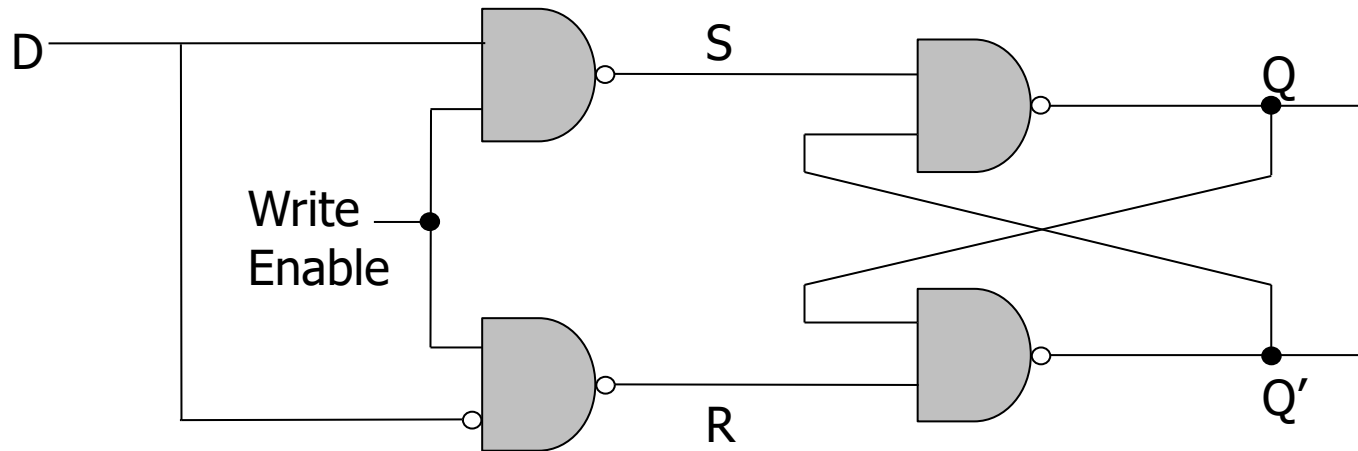
■ **Sequential logic circuits**

- ❑ State & Clock
- ❑ Asynchronous vs. Synchronous

■ **Finite State Machines (FSM)**

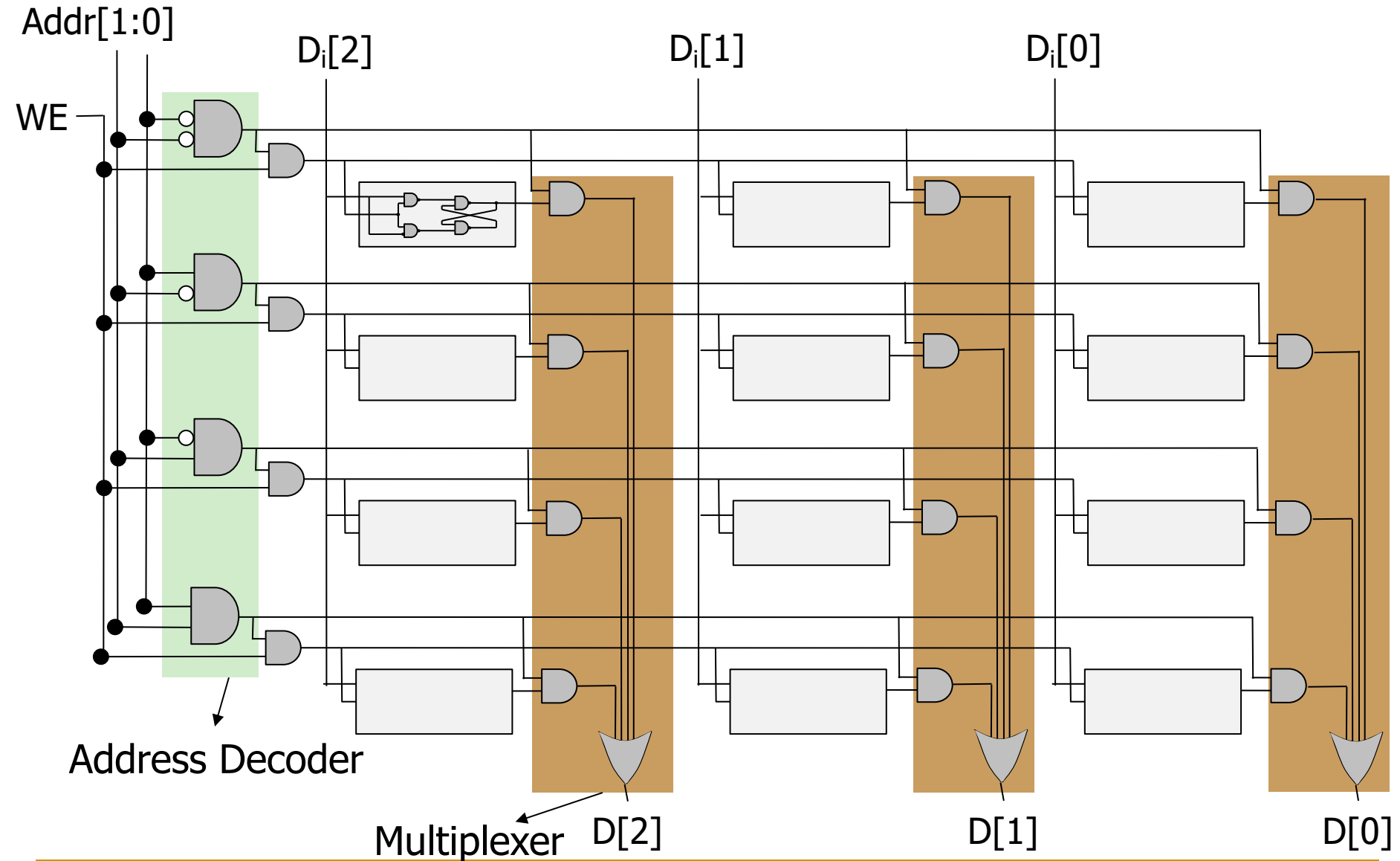
- ❑ How to design FSMs

Recall: The Gated D Latch



Input		Output
WE	D	Q
0	0	Q_{prev}
0	1	Q_{prev}
1	0	0
1	1	1

Recall: A Memory Array (4 locations X 3 bits)



(together w/ decoder, which provides the select input)

Recall: Example: Reading Location 3

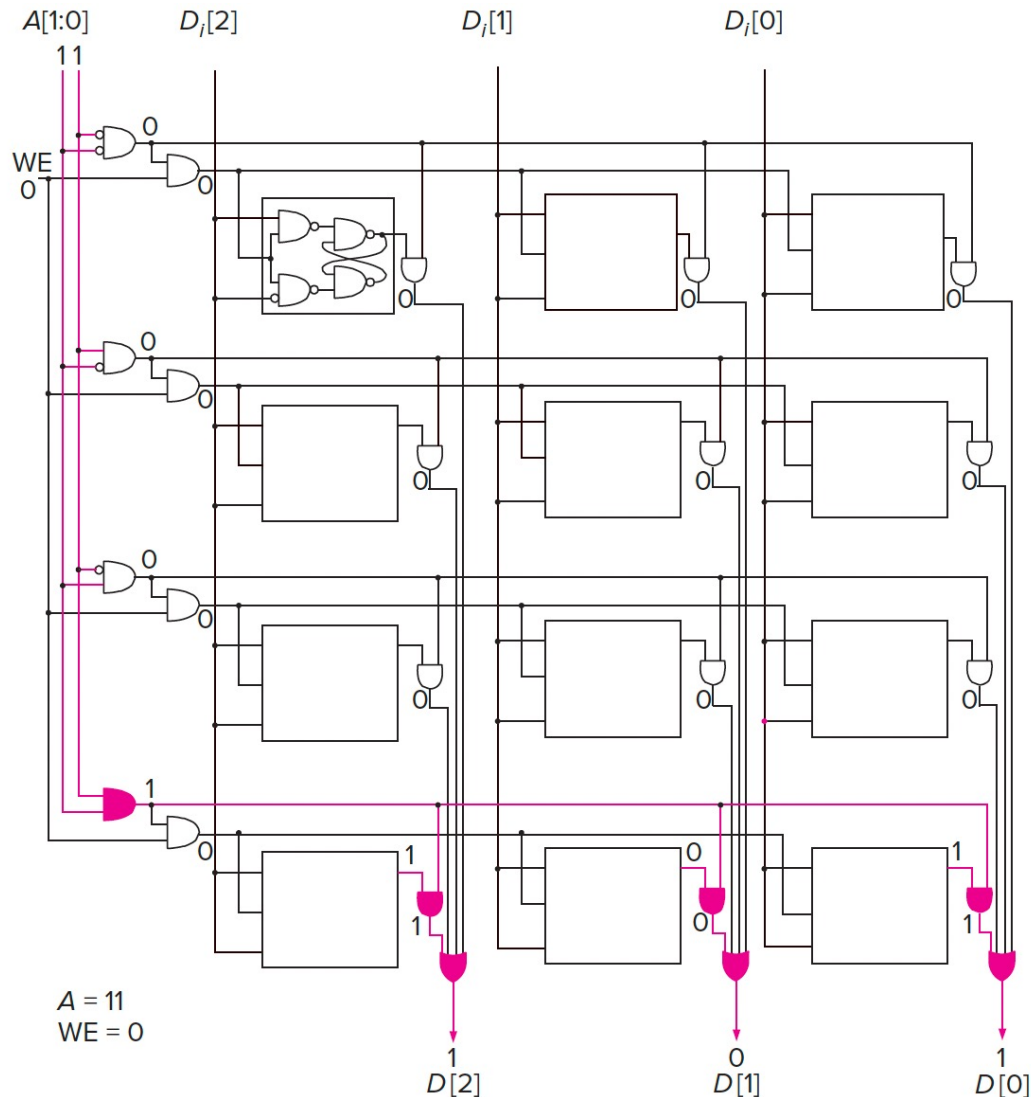


Figure 3.21 Reading location 3 in our 2²-by-3-bit memory.

Sequential Logic Circuits

Recall: Sequential Logic Circuits

- We have examined designs of circuit elements that can **store information**
- Now, we will use these elements to build circuits that **remember** past inputs



Combinational

Only depends on current inputs



Sequential

Opens depending on past inputs

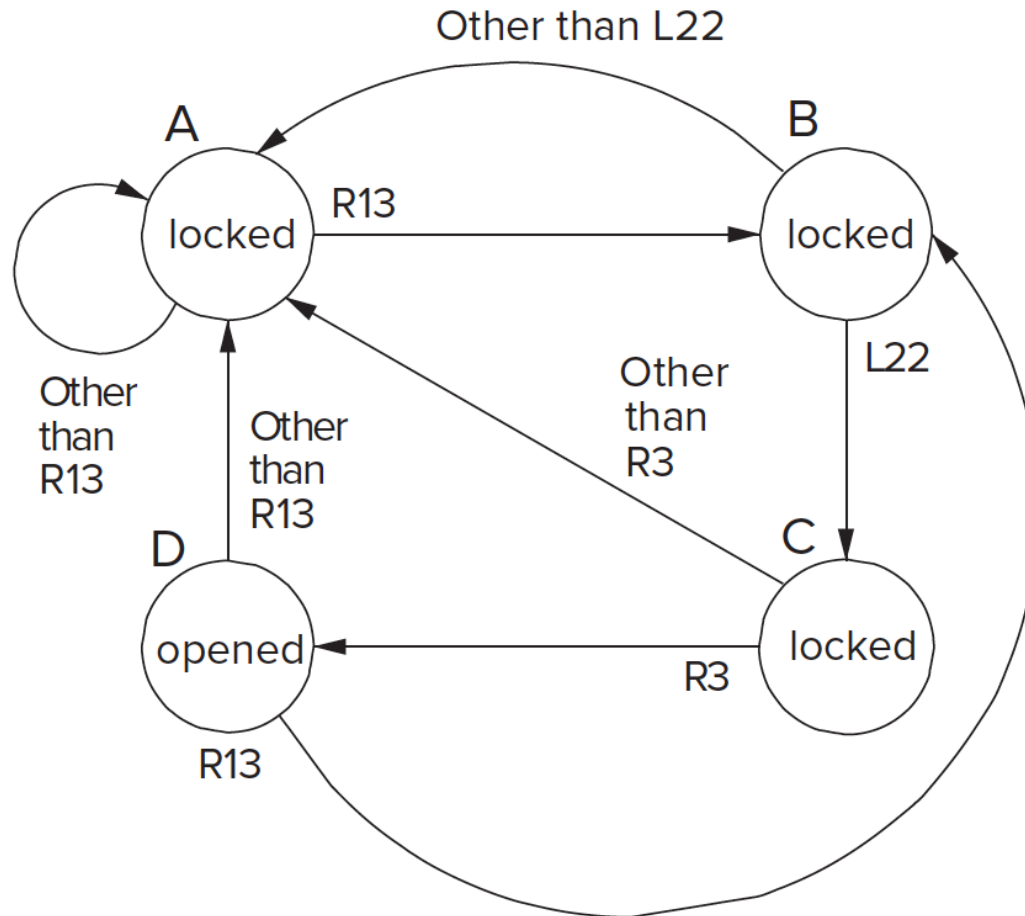
State

- In order for this lock to work, it has to keep track (**remember**) of the past events!
- If passcode is **R13-L22-R3**, sequence of **states** to unlock:
 - A. The lock is not open (locked), and no relevant operations have been performed
 - B. Locked but user has completed R13
 - C. Locked but user has completed R13-L22
 - D. Unlocked: user has completed R13-L22-R3
- The **state** of a system is a snapshot of all relevant elements of the system at the moment of the snapshot
 - ❑ To open the lock, **states A-D must be completed in order**
 - ❑ If anything else happens (e.g., L5), lock **returns** to state A



State Diagram of Our Sequential Lock

- Completely describes the operation of the sequential lock



- We will understand "state diagrams" fully later today

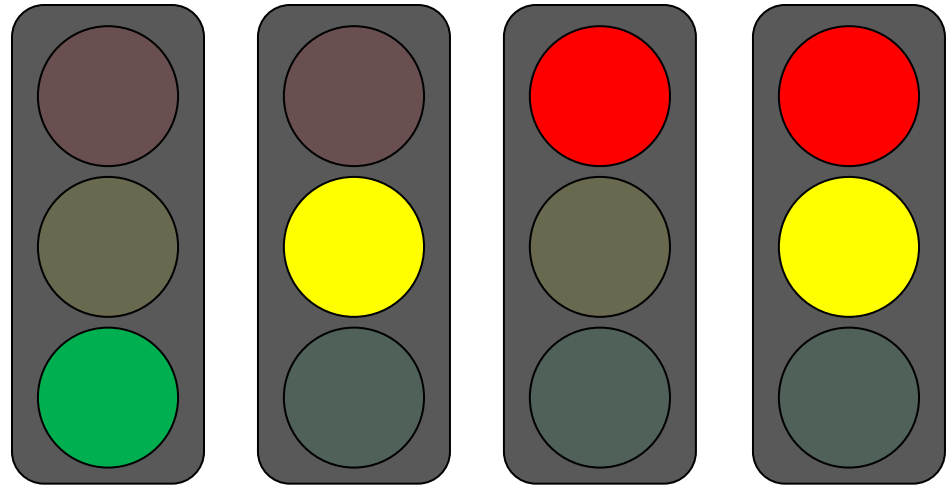
Asynchronous vs. Synchronous State Changes

- Sequential lock we saw is an **asynchronous** “machine”
 - State transitions can take place immediately in response to input
 - There is nothing that synchronizes when each state transition must occur
- Most modern computers are **synchronous** “machines”
 - State transitions take place at fixed units of time (i.e., potentially delayed response to input, synchronized to an external signal)
 - Controlled in part by a clock, as we will see soon
- These are two different design paradigms, with tradeoffs

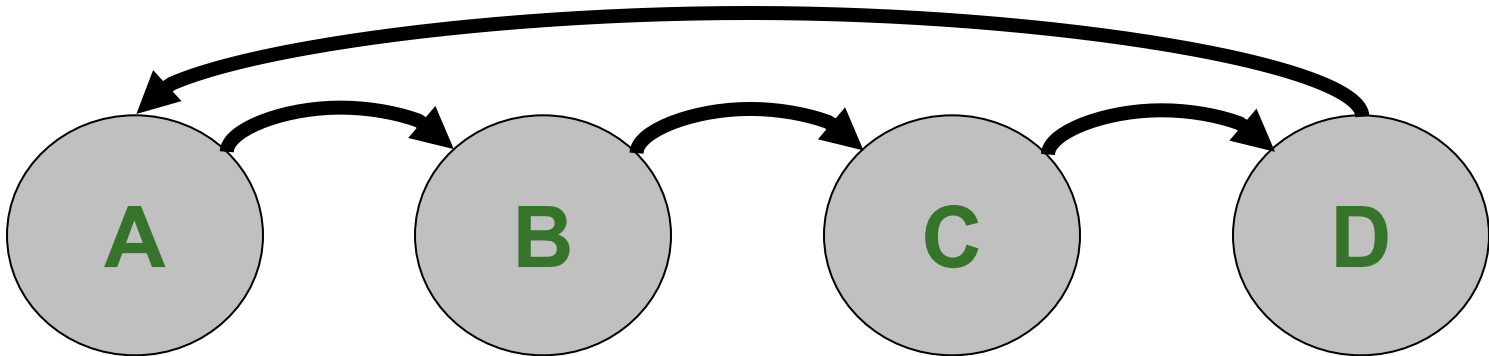
Another Simple Example of State

- A standard Swiss traffic light has **4 states**

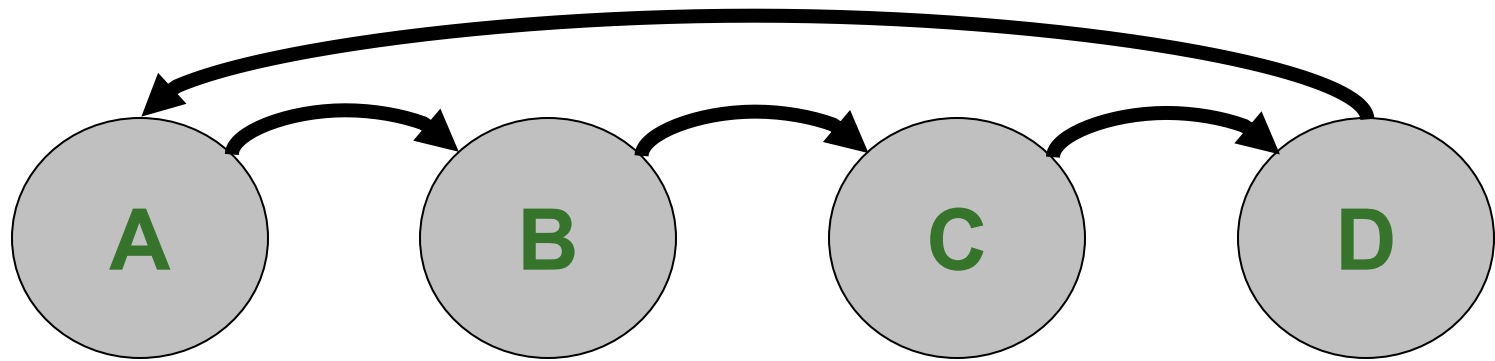
- A. Green
- B. Yellow
- C. Red
- D. Red and Yellow



- The sequence of these states are always as follows



Changing State: The Notion of Clock (I)



- When should the light change from one state to another?
- We need a **clock** to dictate when to change state
 - Clock signal alternates between 0 & 1

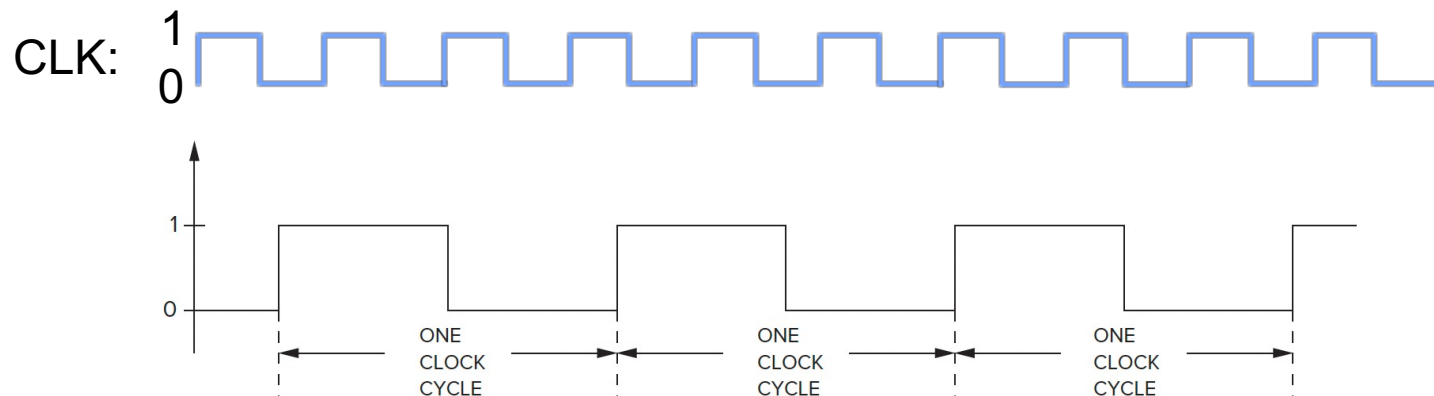
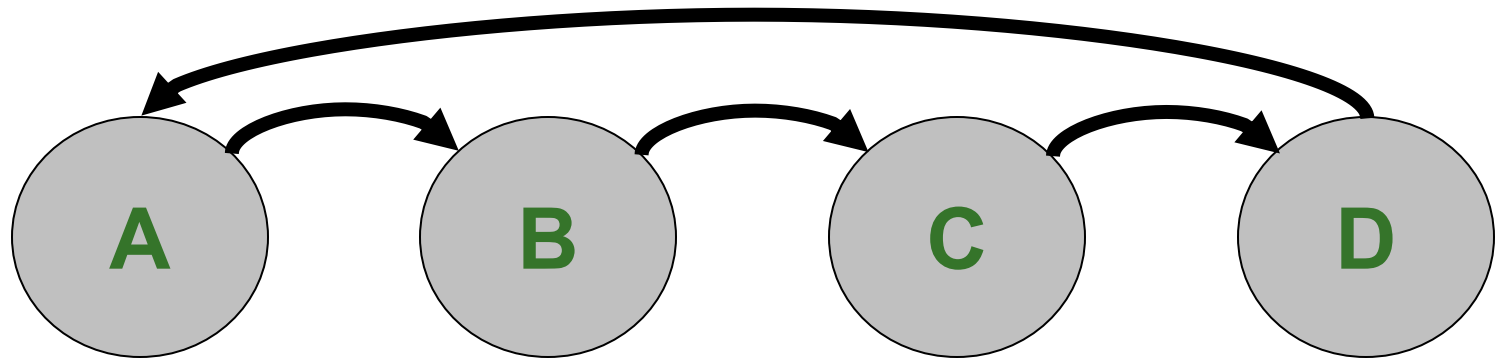


Figure 3.28 A clock signal.

Changing State: The Notion of Clock (I)



- When should the light change from one state to another?
- We need a **clock** to dictate when to change state
 - Clock signal alternates between 0 & 1



- At the start of a clock cycle (), system state changes
 - During a clock cycle, the state stays constant
 - In this traffic light example, we are assuming the traffic light stays in each state an equal amount of time

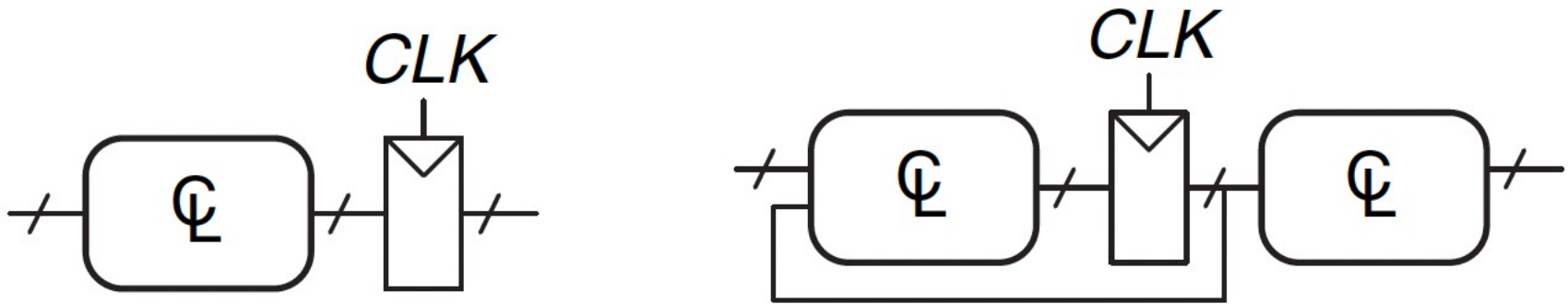
Changing State: The Notion of Clock (II)

- **Clock** is a general mechanism that **triggers transition from one state to another** in a (synchronous) sequential circuit
- Clock **synchronizes state changes** across many sequential circuit elements
- Combinational logic evaluates for the **length of the clock cycle**
- Clock cycle should be chosen to accommodate maximum combinational circuit delay
 - More on this later, when we discuss timing

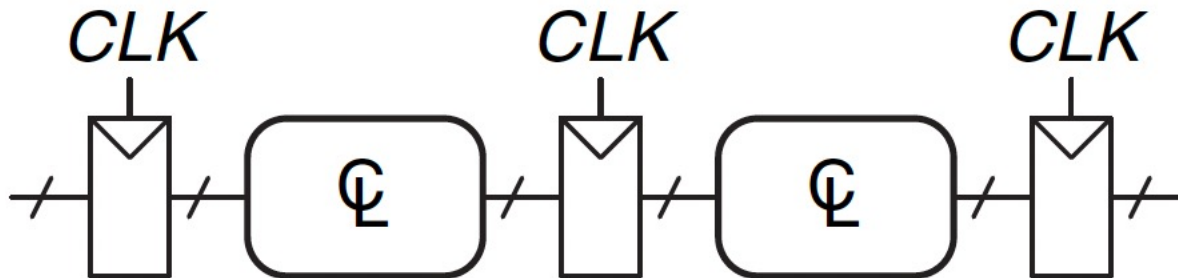
Asynchronous vs. Synchronous State Changes

- Sequential lock we saw is an **asynchronous** “machine”
 - State transitions can take place immediately in response to input
 - There is nothing that synchronizes when each state transition must occur
- Most modern computers are **synchronous** “machines”
 - State transitions take place at fixed units of time (i.e., synchronously)
 - Controlled in part by a clock, as we will see soon
- **These are two different design paradigms, with tradeoffs**
 - Synchronous control can be easier to get correct when the system consists of many components and many states
 - Asynchronous control can be more efficient (no clock overheads)

Synchronous Sequential Circuits



Major types of Sequential Logic Circuits



Finite State Machines

Finite State Machines

- What is a **Finite State Machine** (FSM)?
 - **A discrete-time model** of a stateful system
 - Each state represents a snapshot of the system at a given time
- An FSM pictorially shows
 1. the set of all possible **states** that a system can be in
 2. how the system transitions from one state to another
- An FSM can model
 - A traffic light, an elevator, fan speed, a microprocessor, etc.
- **An FSM enables us to pictorially think of a stateful system using simple diagrams**

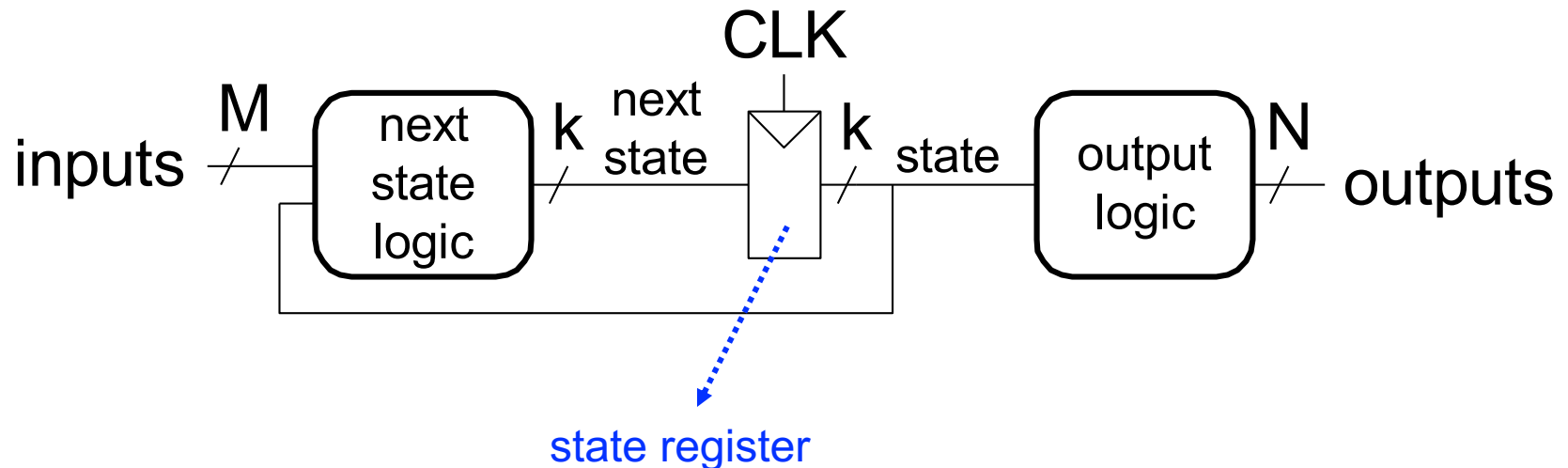
Finite State Machines (FSMs) Consist of:

■ Five elements:

1. A **finite** number of **states**
 - **State**: snapshot of all relevant elements of the system at the time of the snapshot
2. A **finite** number of external **inputs**
3. A **finite** number of external **outputs**
4. An explicit **specification of all state transitions**
 - How to get from one state to another
5. An explicit **specification of what determines each external output value**

Finite State Machines (FSMs)

- Each FSM consists of three separate parts:
 - next state logic
 - state register
 - output logic

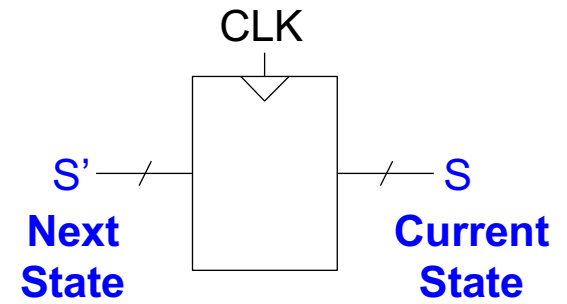


At the beginning of the clock cycle, next state is latched into the state register

Finite State Machines (FSMs) Consist of:

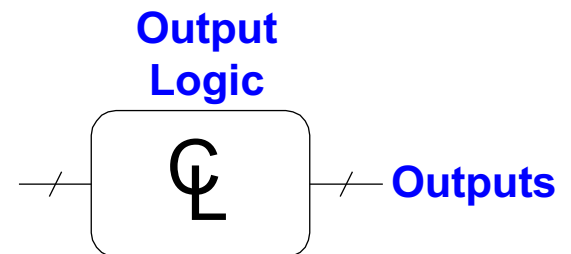
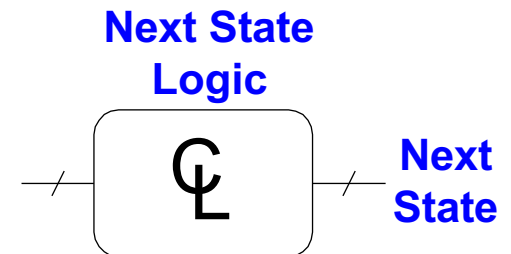
■ Sequential Circuits

- State register(s)
 - Store the current state and
 - Provide the next state at the clock edge



■ Combinational Circuits

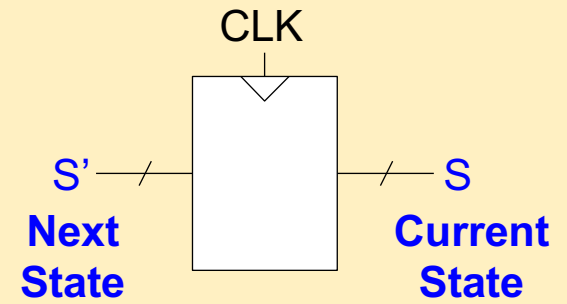
- Next state logic
 - Determines what the next state will be
- Output logic
 - Generates the outputs



Finite State Machines (FSMs) Consist of:

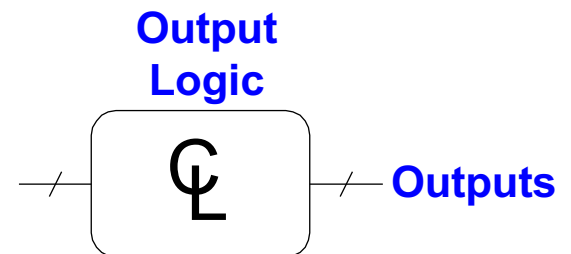
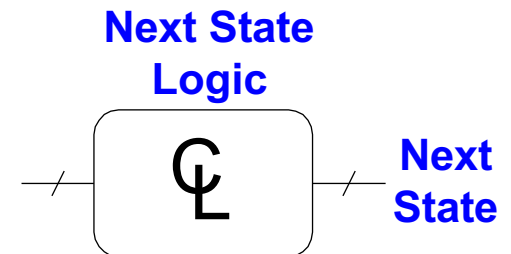
■ Sequential Circuits

- State register(s)
 - Store the current state and
 - Provide the next state at the clock edge



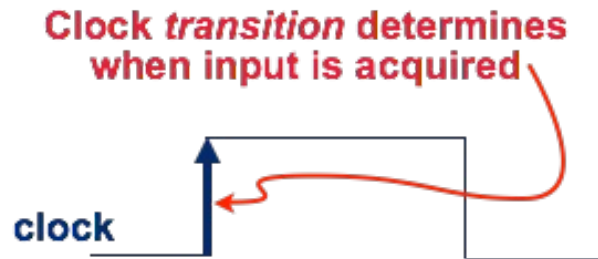
■ Combinational Circuits

- Next state logic
 - Determines what the next state will be
- Output logic
 - Generates the outputs

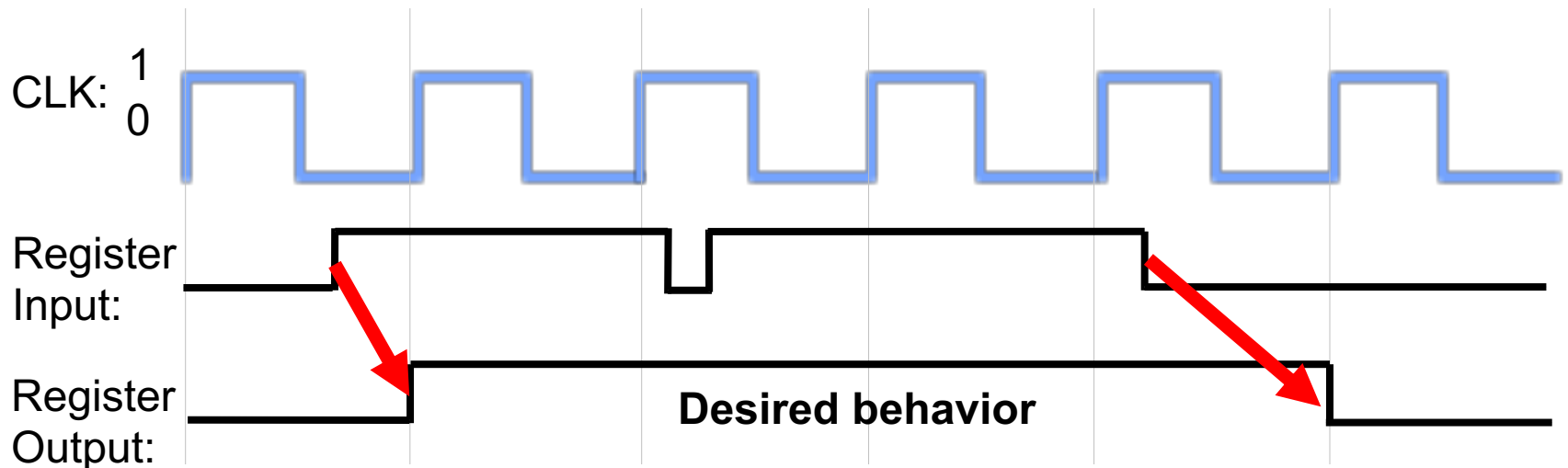


State Register Implementation

- How can we implement a **state register**? Two properties:
 1. We need to store data at the **beginning** of every clock cycle

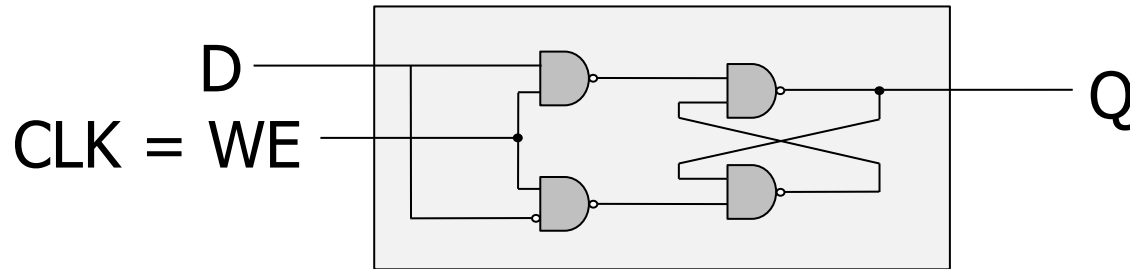


2. The data must be **available** during the **entire clock cycle**

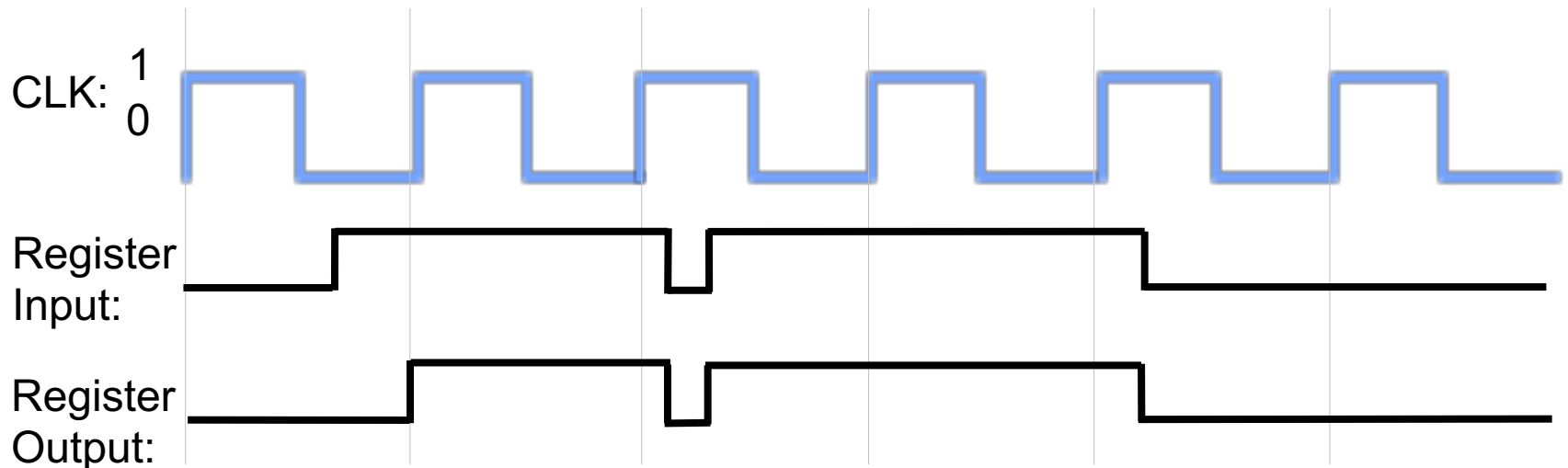


The Problem with Latches: Transparency

Recall the
Gated D Latch

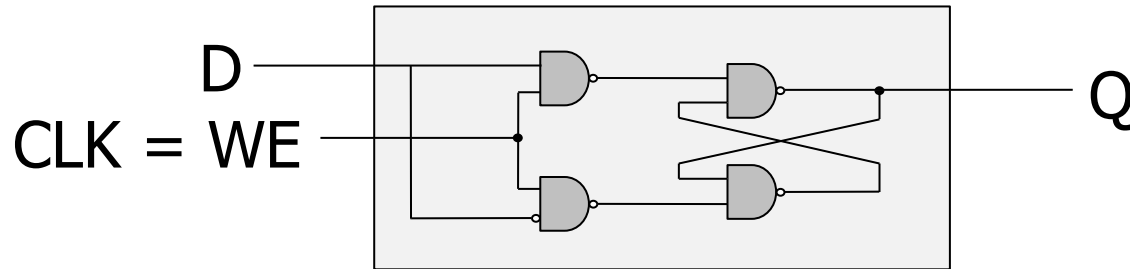


- Currently, we **cannot** simply wire a clock to WE of a latch
 - ❑ **Whenever the clock is high**, the latch propagates **D** to **Q**
 - ❑ **The latch is “transparent”**

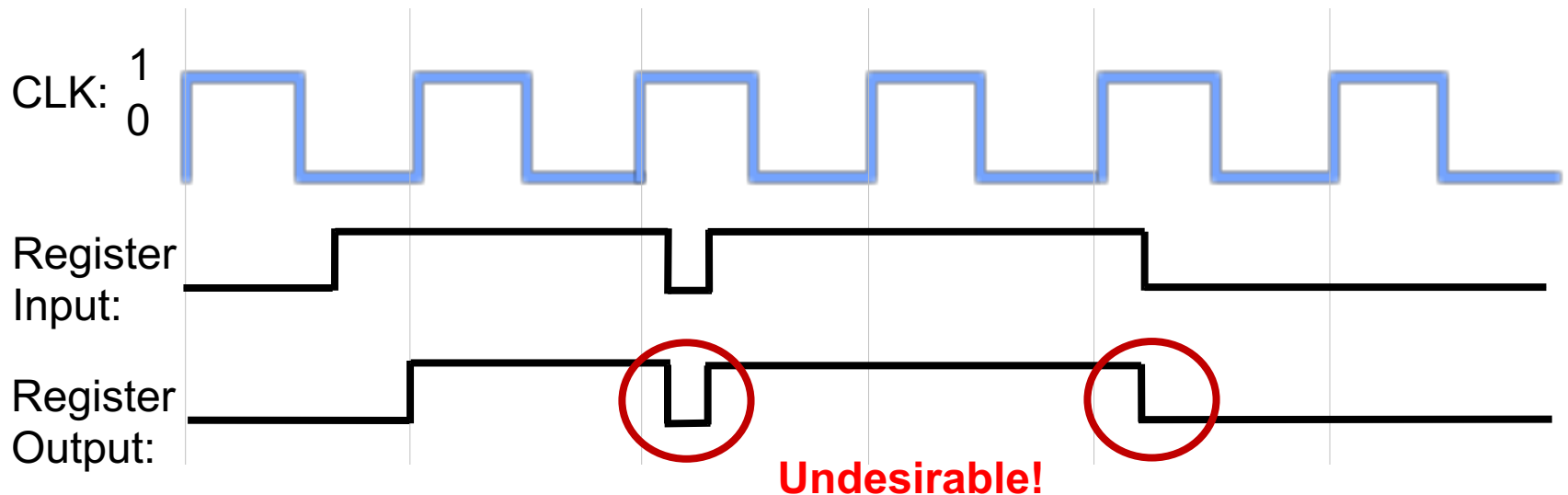


The Problem with Latches: Transparency

Recall the
Gated D Latch

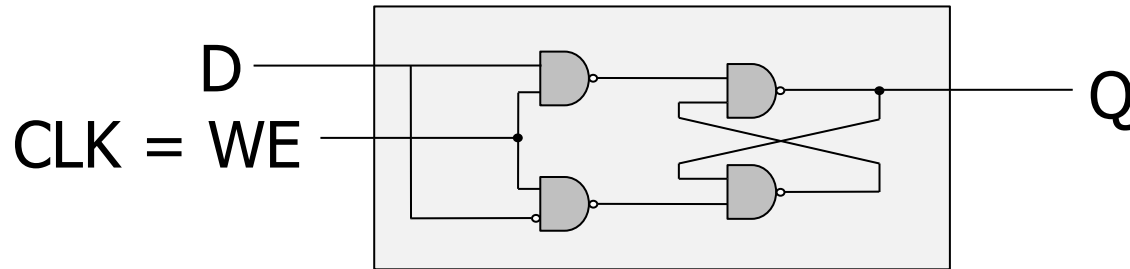


- Currently, we **cannot** simply wire a clock to WE of a latch
 - ❑ **Whenever the clock is high**, the latch propagates **D** to **Q**
 - ❑ **The latch is “transparent”**



The Problem with Latches: Transparency

Recall the
Gated D Latch



How can we change the latch, so that

1) D (input) is **observable** at **Q** (output)
only at the **beginning of next** clock cycle?

2) Q is **available for the full clock cycle**

The Need for a New Storage Element

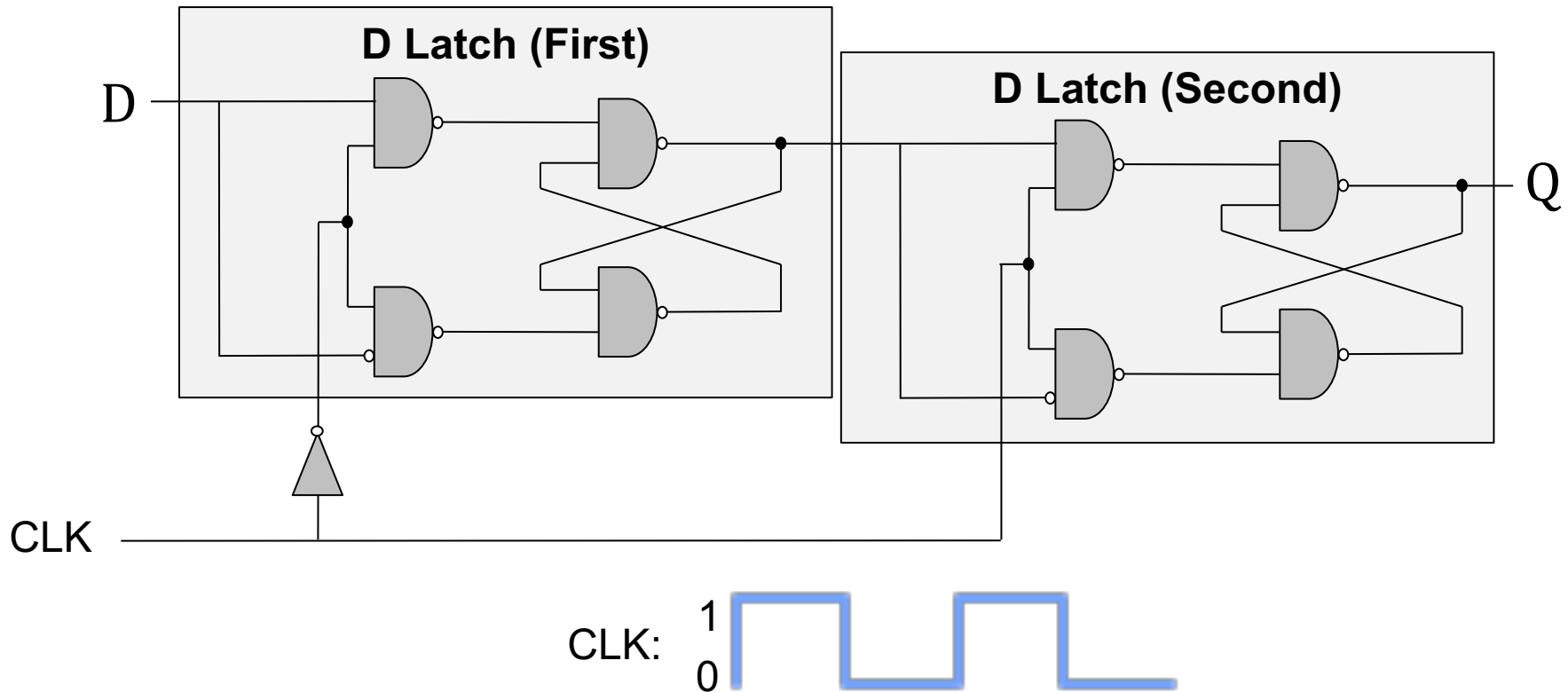
- To design viable FSMs
- We need storage elements that allow us to:
 - **read the current state** throughout the **entire current clock cycle**

AND

- **not write the next state** values into the storage elements **until** the beginning of the **next clock cycle**

The D Flip-Flop

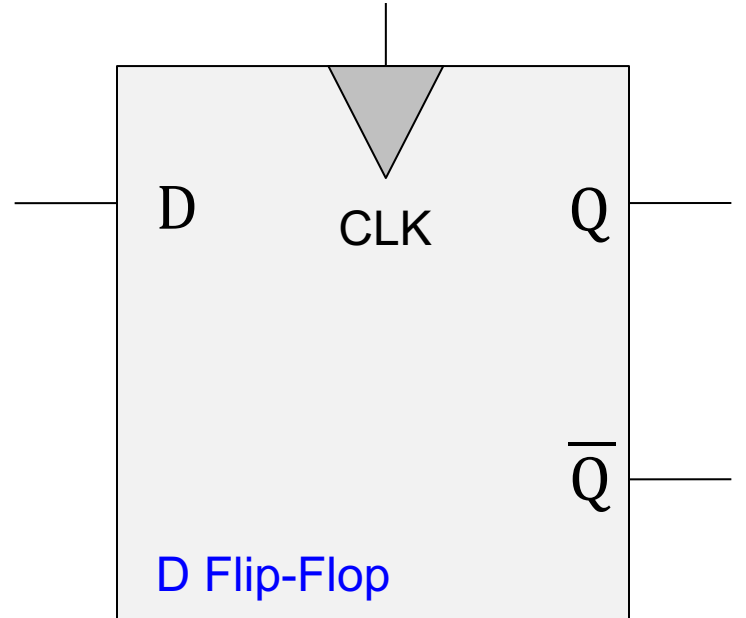
- 1) state change on clock edge, 2) data available for full cycle



- When the clock is low, 1st latch propagates **D** to the input of the 2nd (Q unchanged)
- Only when the clock is high, 2nd latch latches **D** (**Q stores D**)
 - At the rising edge of clock (clock going from 0→1), Q gets assigned D

The D Flip-Flop

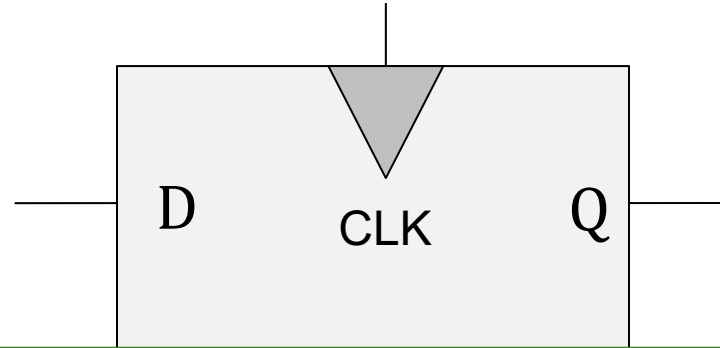
- 1) state change on clock edge, 2) data available for full cycle



- At the rising edge of clock (clock going from 0→1), **Q** gets assigned **D**
- At all other times, Q is unchanged

The D Flip-Flop

- 1) state change on clock edge, 2) data available for full cycle



We can use **D Flip-Flops**
to implement the state register

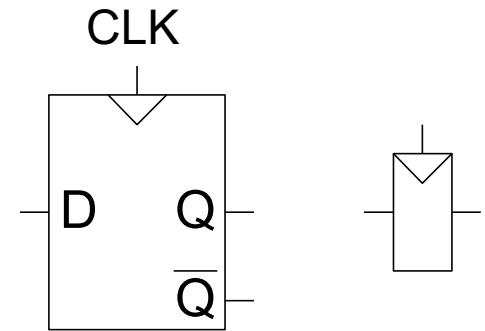
- At the rising edge of clock (clock going from 0→1), **Q** gets assigned **D**
- At all other times, **Q** is unchanged

Rising-Clock-Edge Triggered Flip-Flop

- **Two inputs:** CLK, D

- **Function**

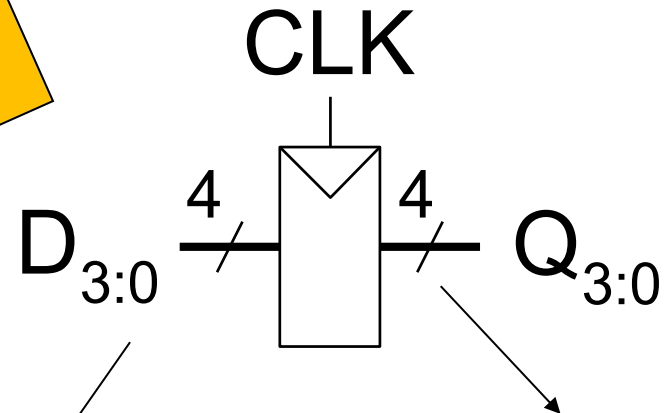
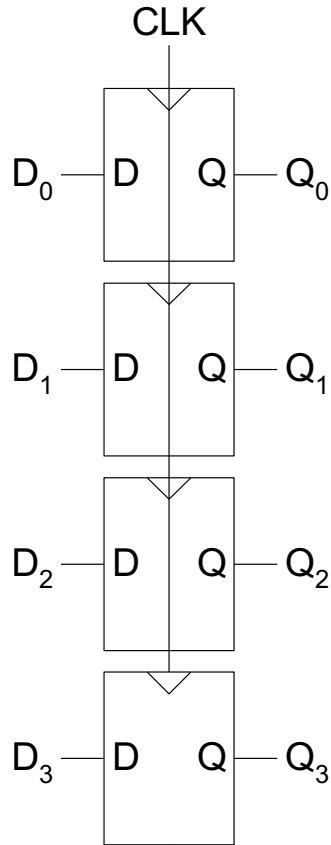
- The flip-flop “samples” **D** on the rising edge of CLK (**positive edge**)
- When CLK rises from 0 to 1, **D** passes through to **Q**
- Otherwise, **Q** holds its previous value
- **Q** changes **only** on the rising edge of CLK



- A flip-flop is called an **edge-triggered state element** because it captures data on the clock edge
 - A latch is a **level-triggered** state element

D Flip-Flop Based Register

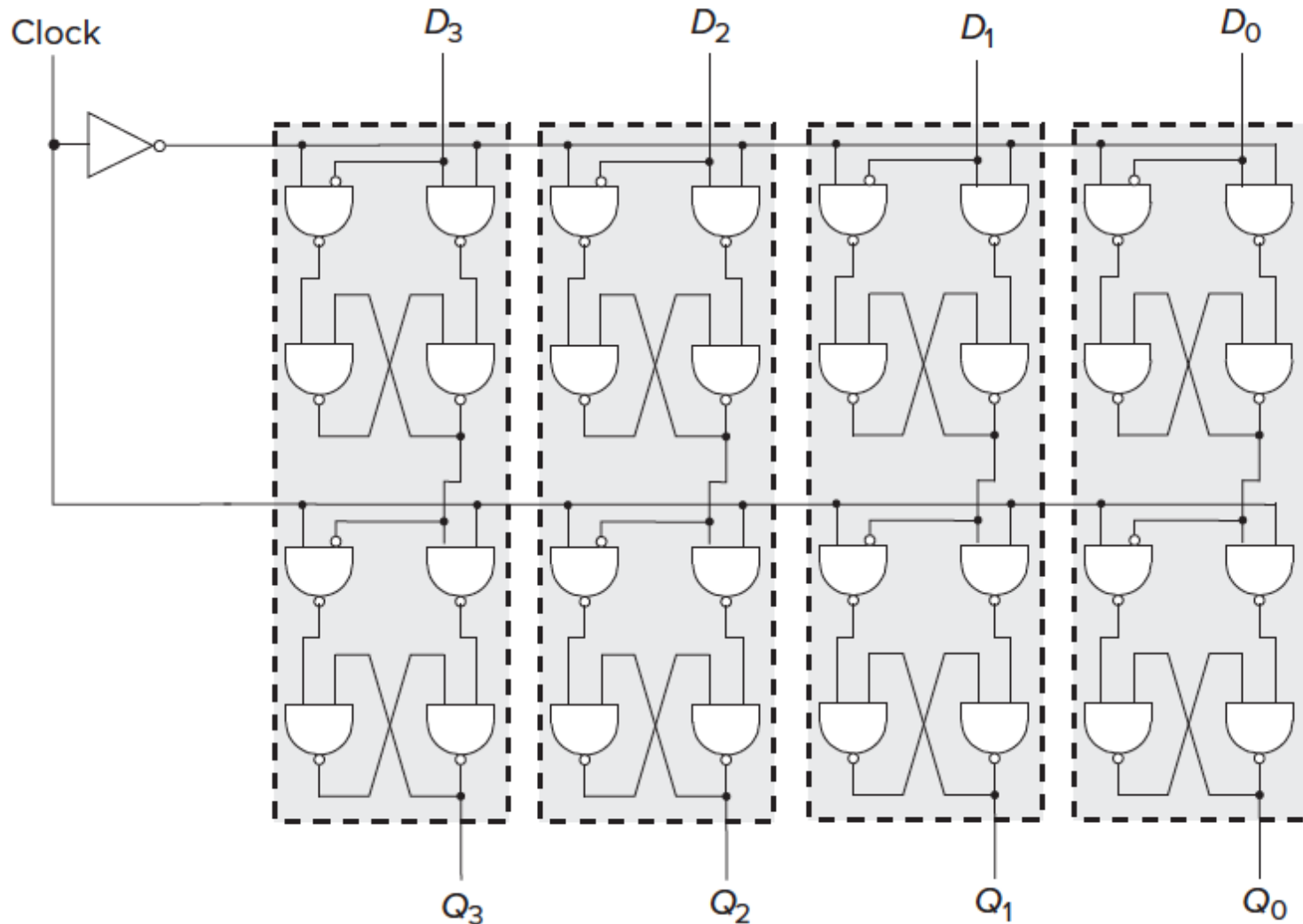
- Multiple parallel D flip-flops, each of which storing 1 bit



This line represents 4 wires

This register stores 4 bits

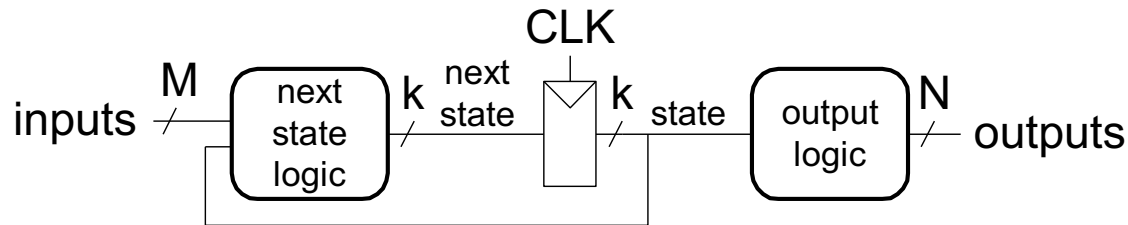
A 4-Bit D-Flip-Flop-Based Register (Internally)



Finite State Machines (FSMs)

- Next state is determined by the current state and the inputs
- Two types of finite state machines differ in the **output logic**:
 - **Moore FSM**: outputs depend only on the current state

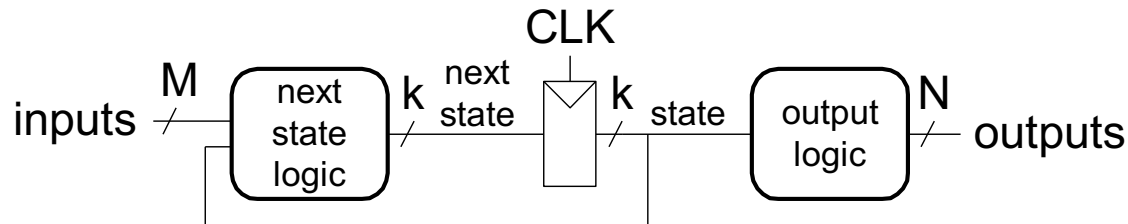
Moore FSM



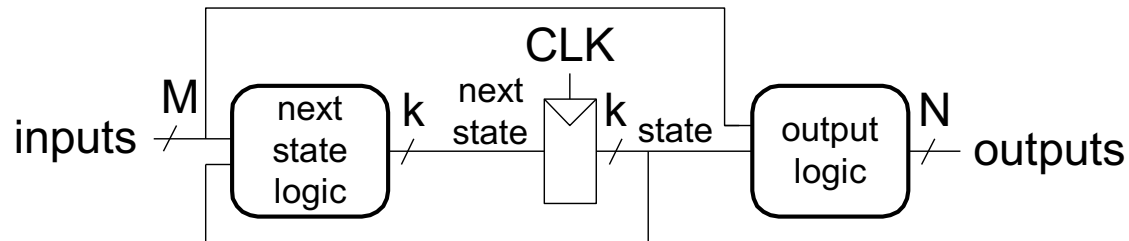
Finite State Machines (FSMs)

- Next state is determined by the current state and the inputs
- Two types of finite state machines differ in the **output logic**:
 - **Moore FSM**: outputs depend only on the current state
 - **Mealy FSM**: outputs depend on the current state and the inputs

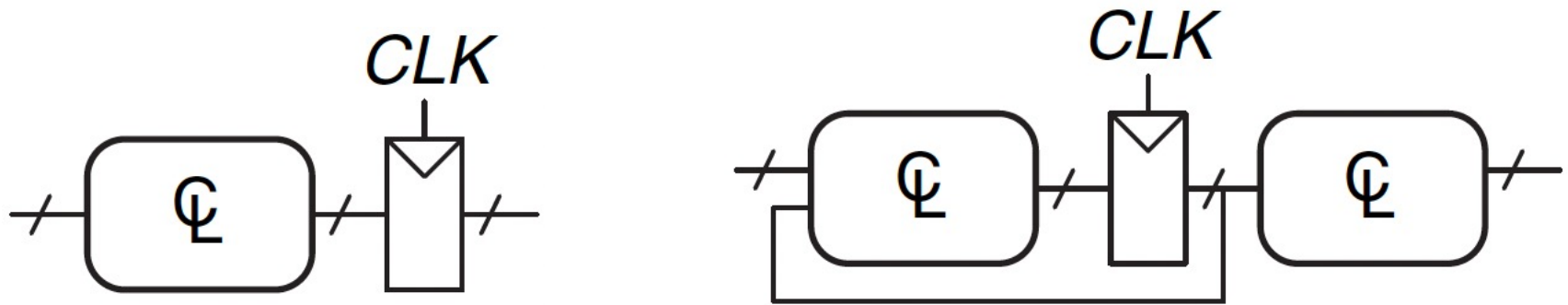
Moore FSM



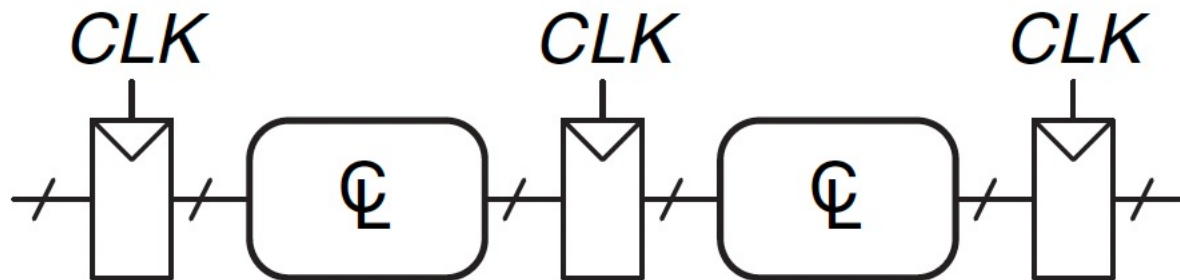
Mealy FSM



Recall: Synchronous Sequential Circuits

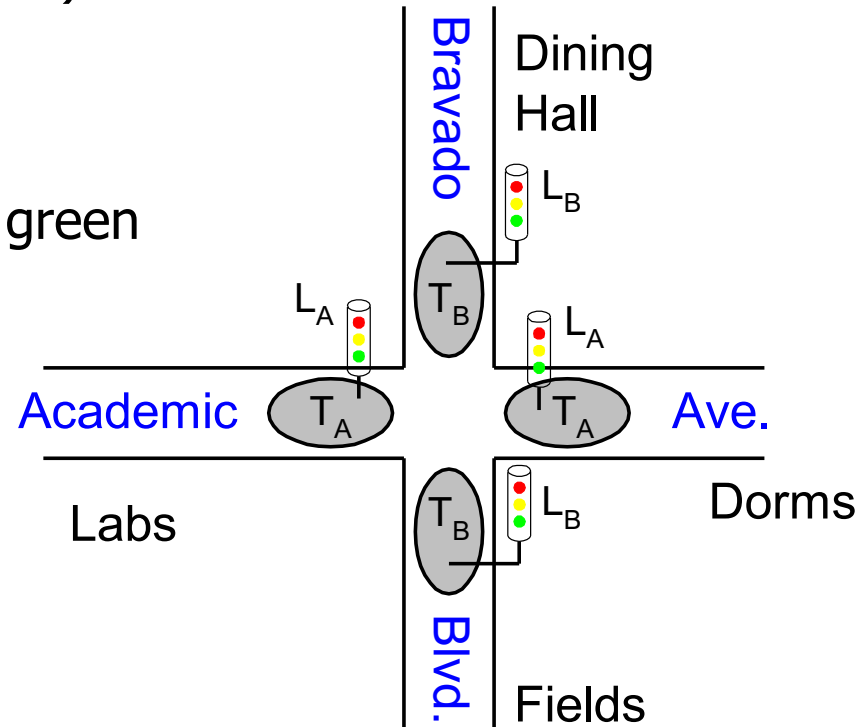


Major types of Sequential Logic Circuits



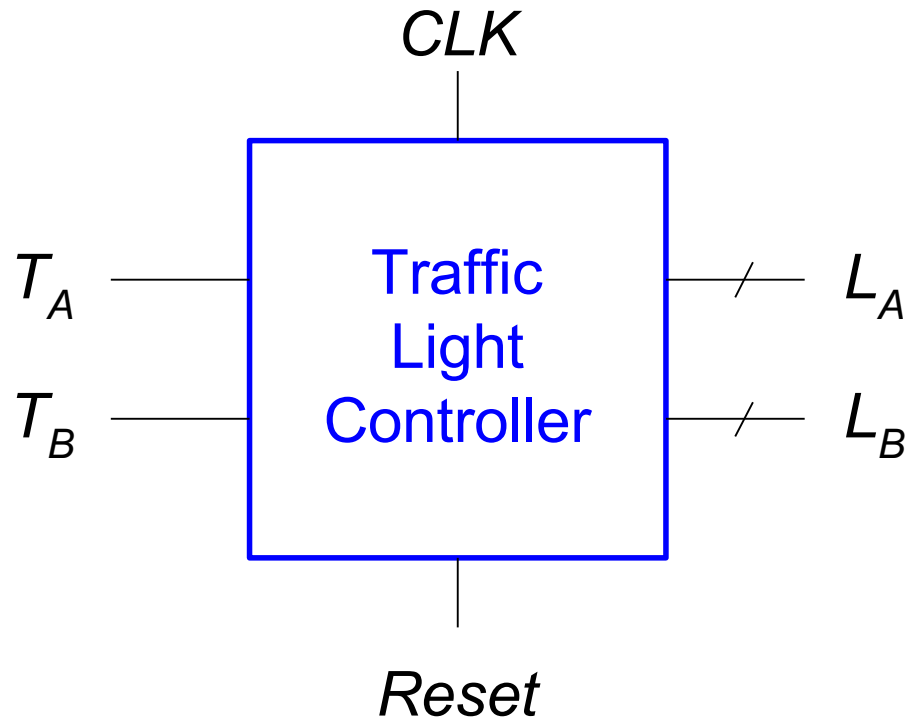
Finite State Machine Example

- “Smart” traffic light controller
 - **2 inputs:**
 - Traffic sensors: T_A , T_B (TRUE when there’s traffic)
 - **2 outputs:**
 - Lights: L_A , L_B (Red, Yellow, Green)
 - State changes every 5 seconds
 - Except if green and traffic, stay green



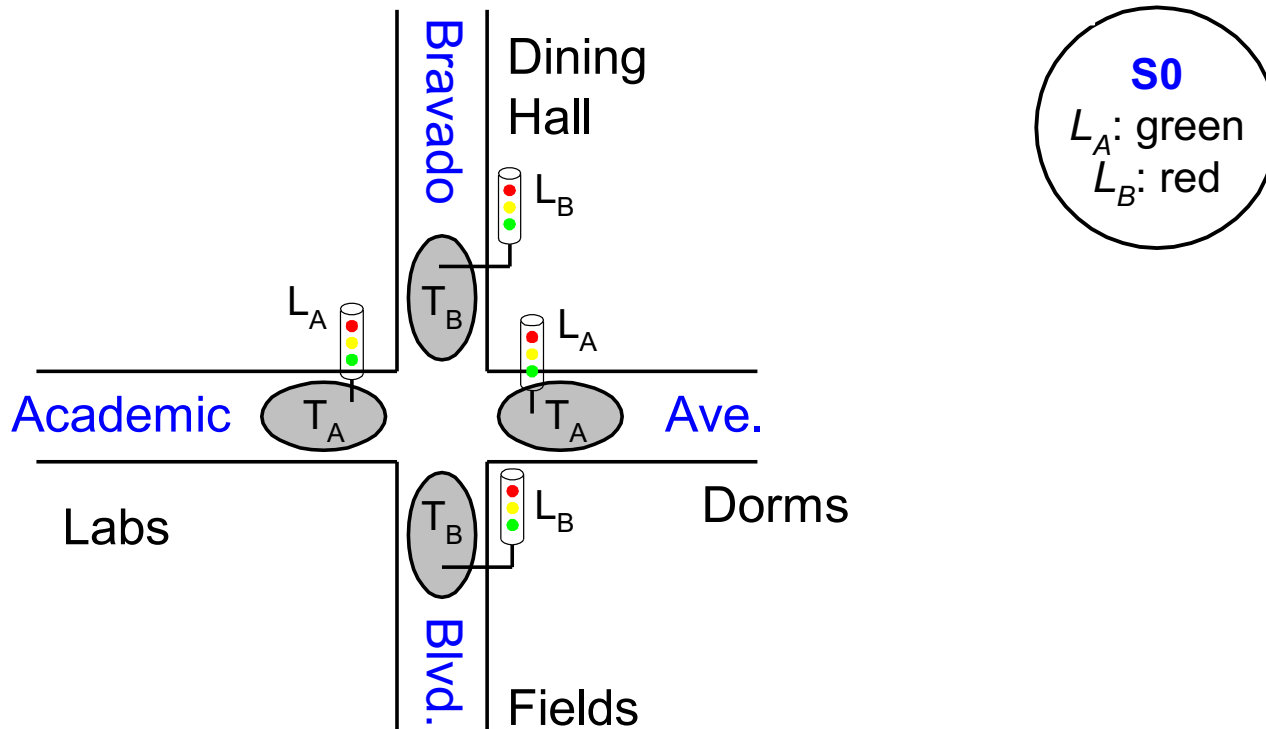
Finite State Machine Black Box

- **Inputs:** CLK, Reset, T_A , T_B
- **Outputs:** L_A , L_B



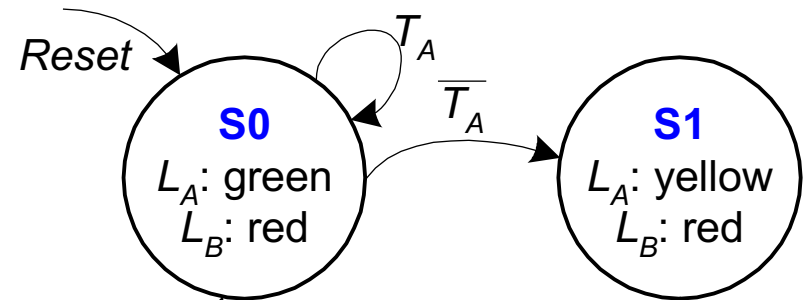
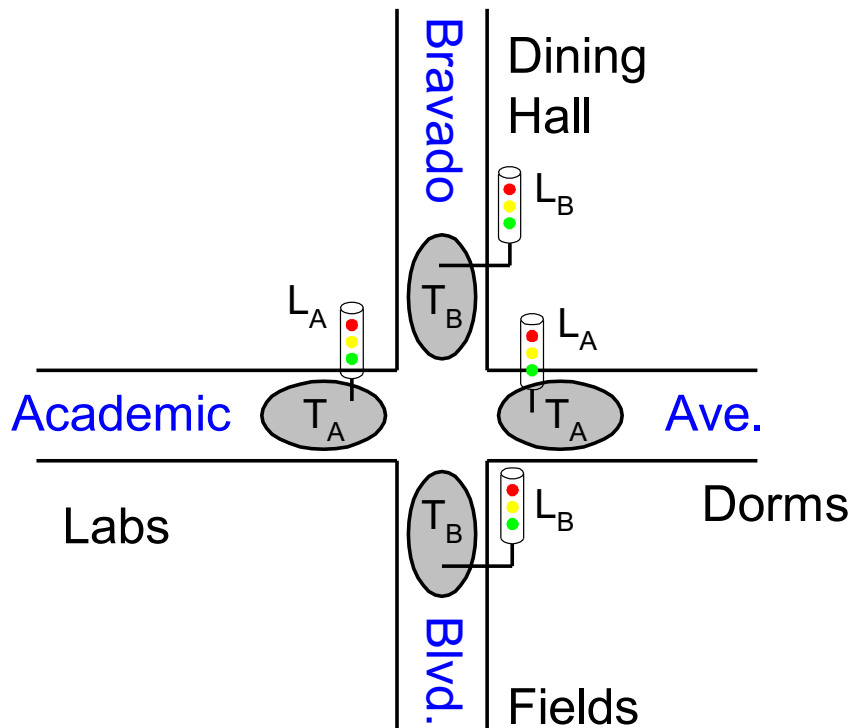
Finite State Machine Transition Diagram

- **Moore FSM:** outputs labeled in each state
 - **States:** Circles
 - **Transitions:** Arcs



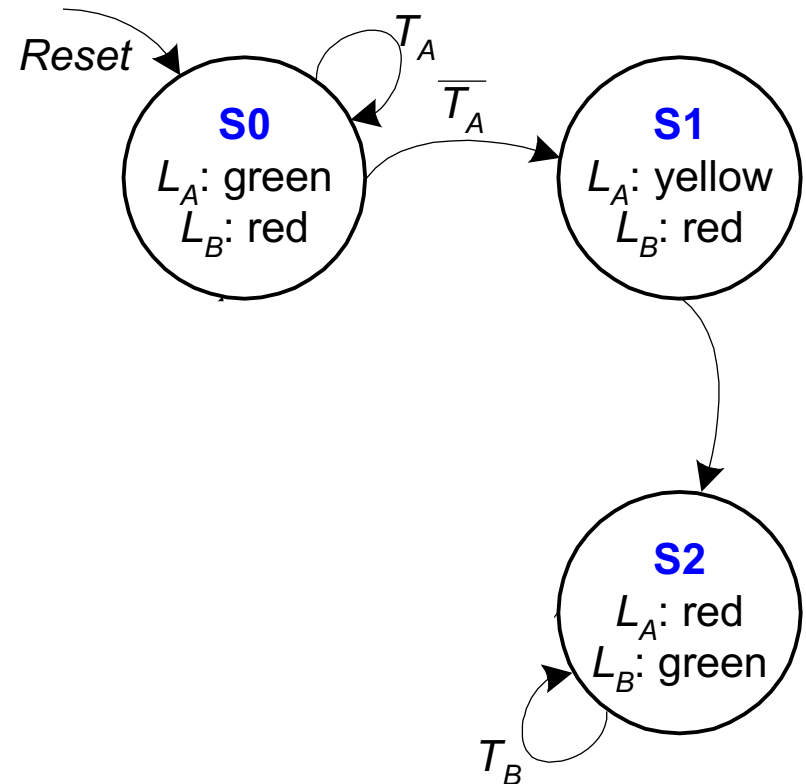
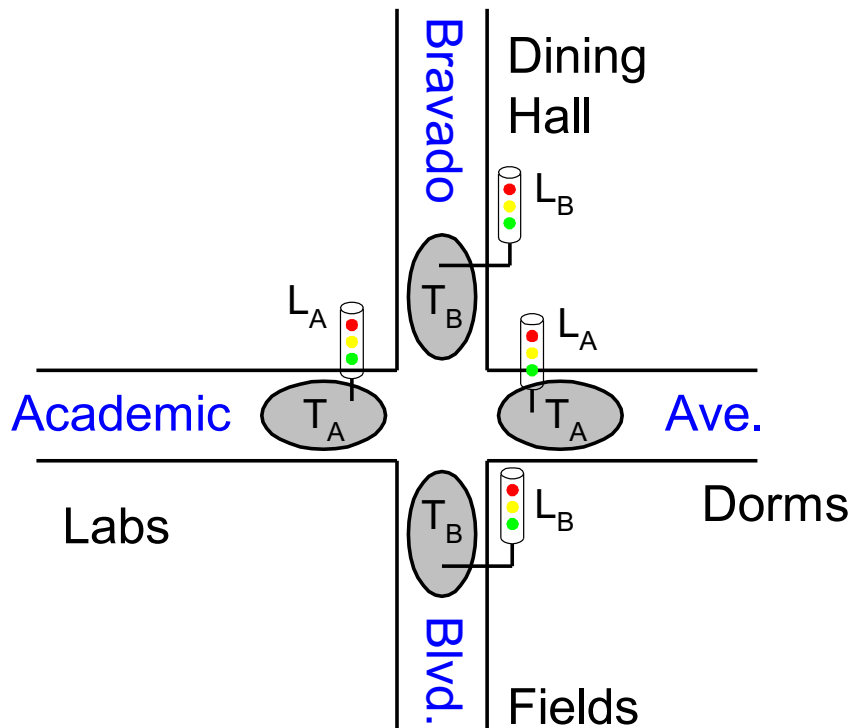
Finite State Machine Transition Diagram

- **Moore FSM:** outputs labeled in each state
 - **States:** Circles
 - **Transitions:** Arcs



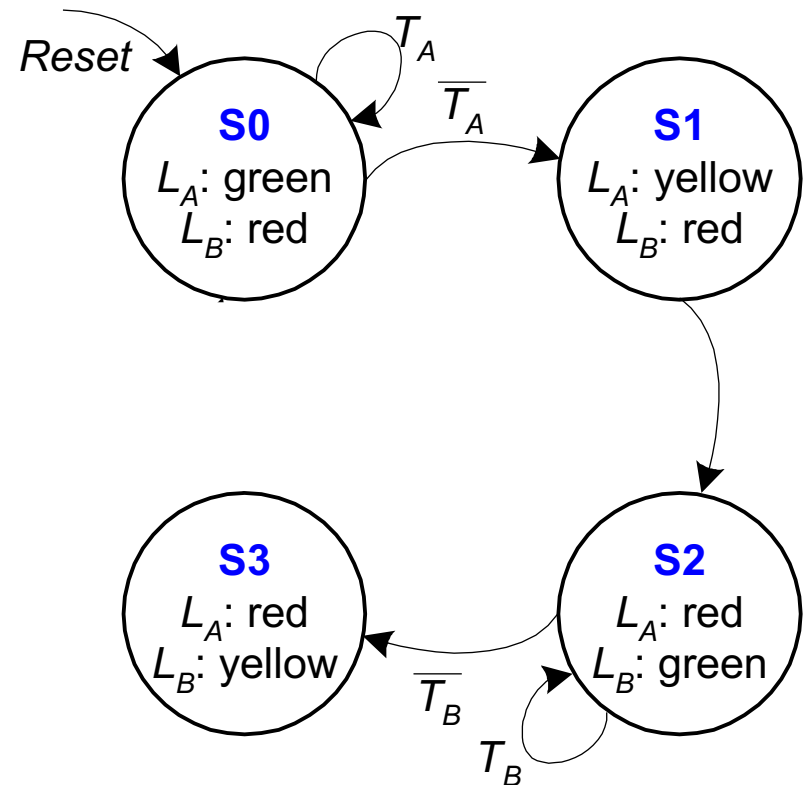
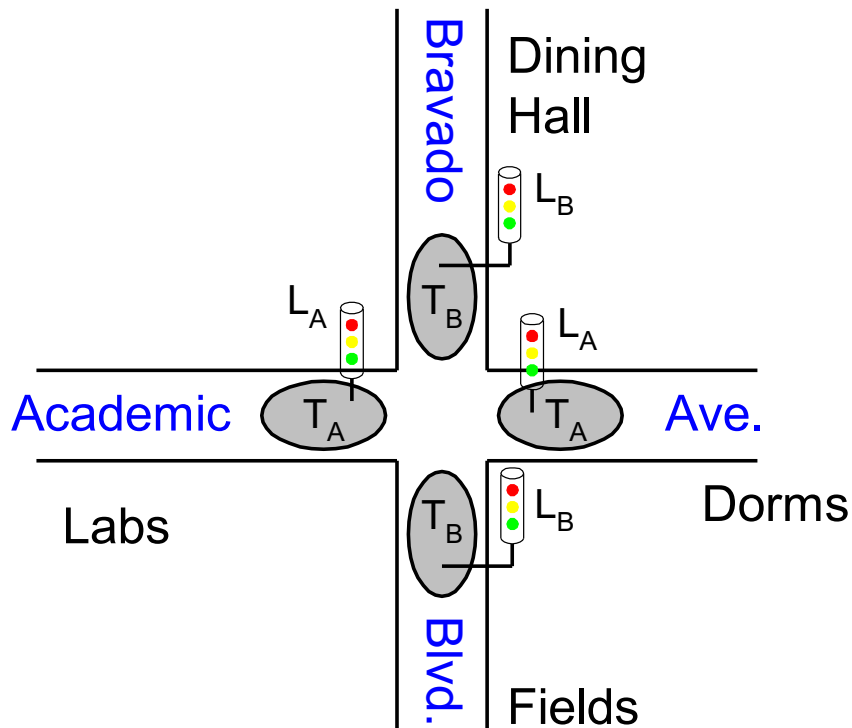
Finite State Machine Transition Diagram

- **Moore FSM:** outputs labeled in each state
 - **States:** Circles
 - **Transitions:** Arcs



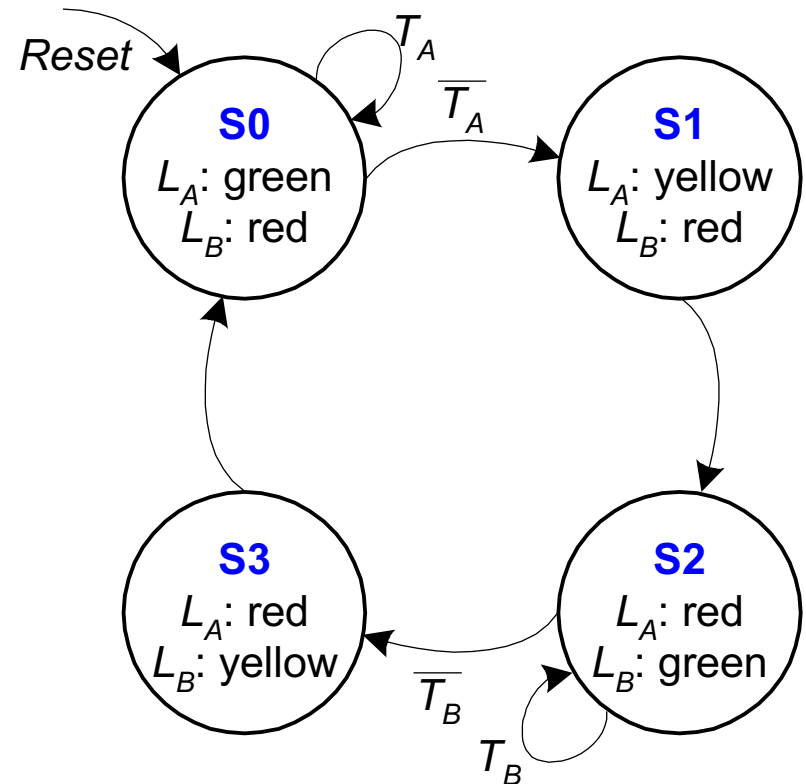
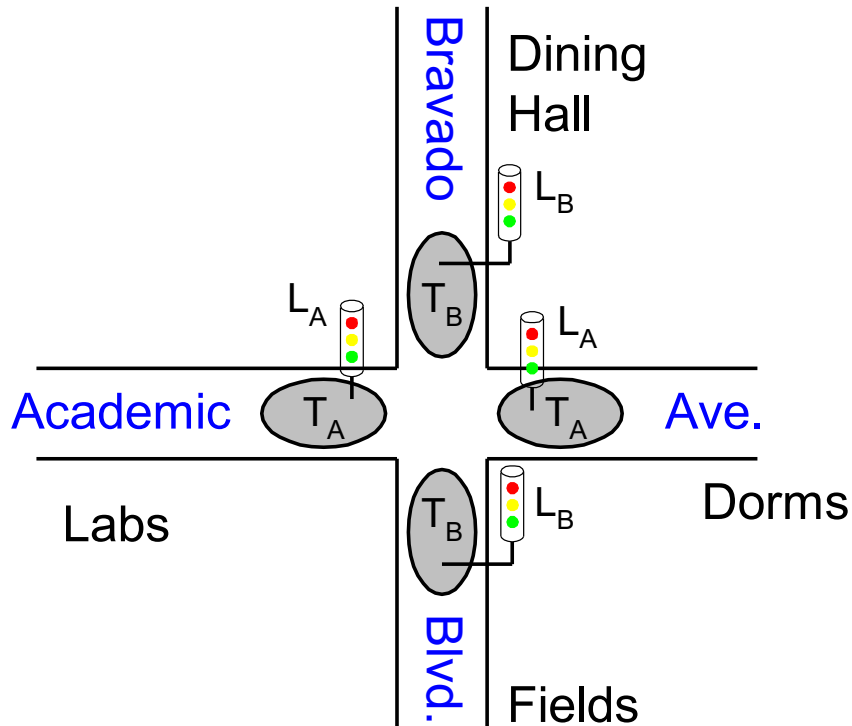
Finite State Machine Transition Diagram

- **Moore FSM:** outputs labeled in each state
 - **States:** Circles
 - **Transitions:** Arcs



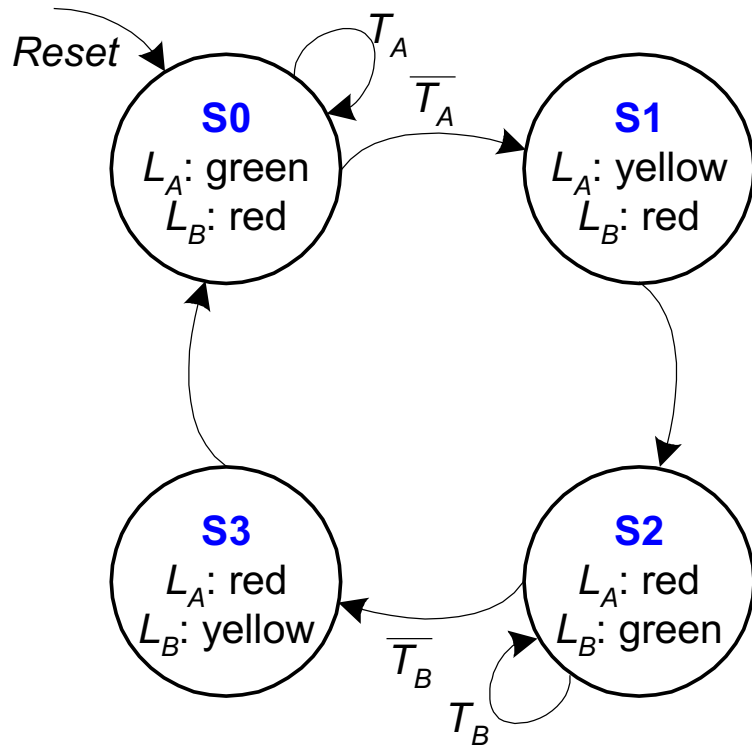
Finite State Machine Transition Diagram

- **Moore FSM:** outputs labeled in each state
 - **States:** Circles
 - **Transitions:** Arcs



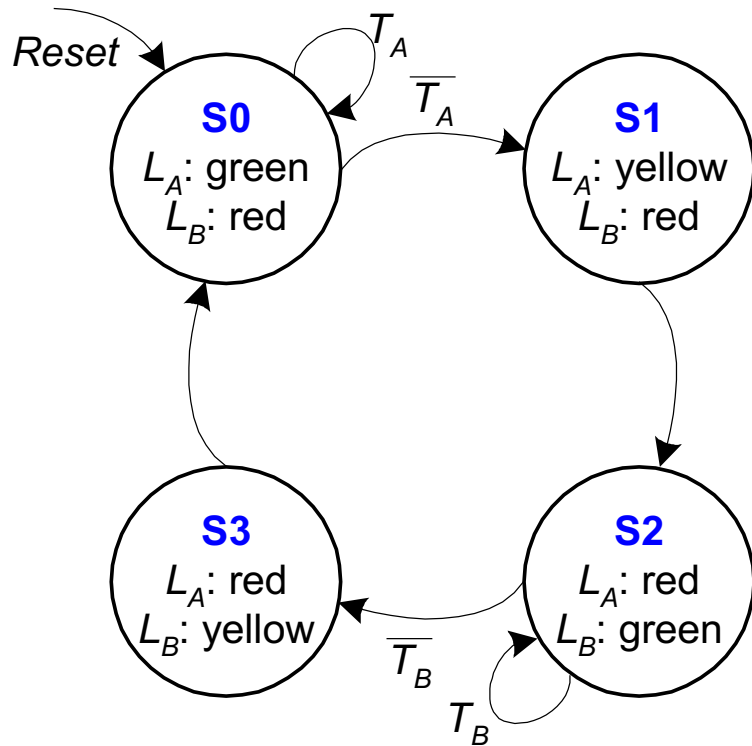
Finite State Machine: State Transition Table

FSM State Transition Table



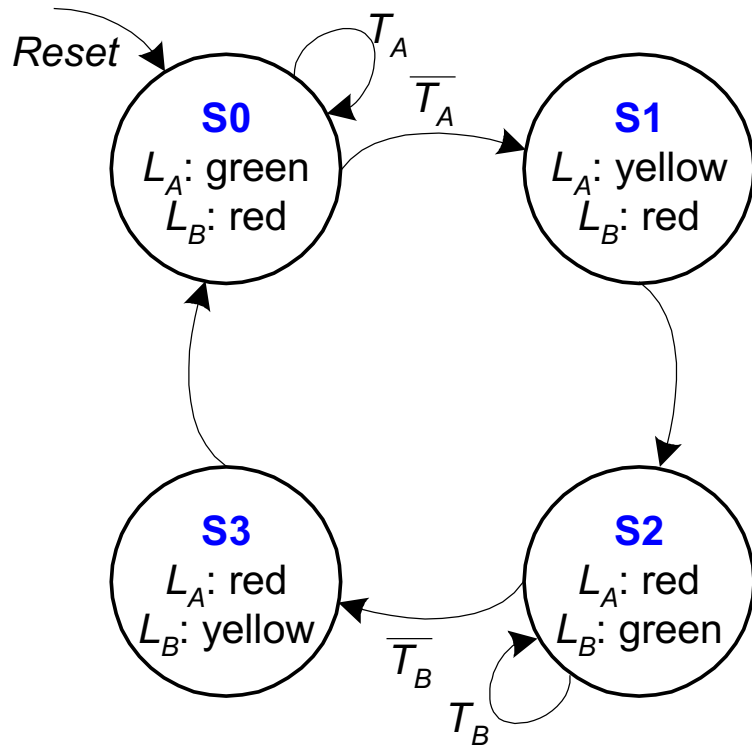
Current State	Inputs		Next State
S	T_A	T_B	S'
S0	0	X	
S0	1	X	
S1	X	X	
S2	X	0	
S2	X	1	
S3	X	X	

FSM State Transition Table



Current State	Inputs		Next State
S	T_A	T_B	S'
S0	0	X	S1
S0	1	X	S0
S1	X	X	S2
S2	X	0	S3
S2	X	1	S2
S3	X	X	S0

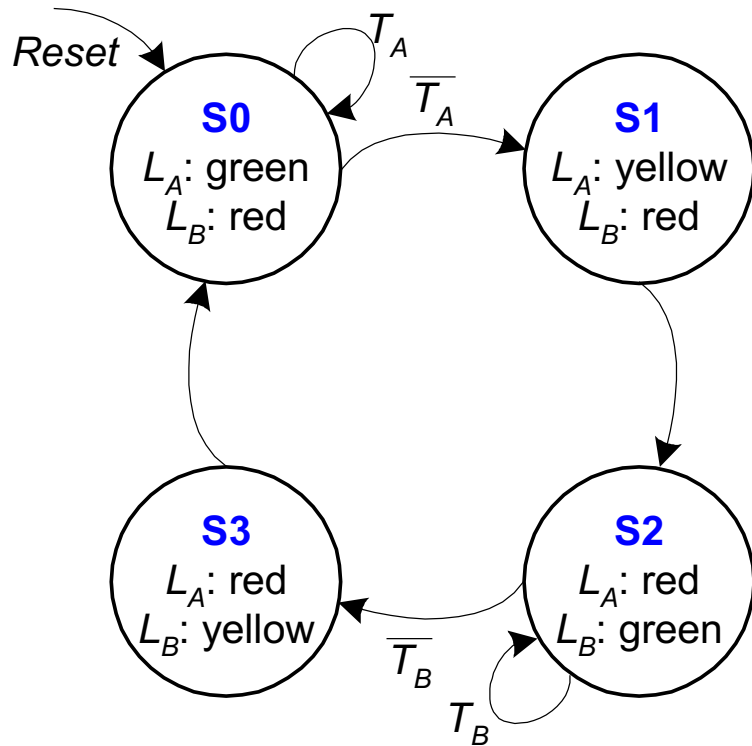
FSM State Transition Table



Current State	Inputs		Next State
S	T_A	T_B	S'
S0	0	X	S1
S0	1	X	S0
S1	X	X	S2
S2	X	0	S3
S2	X	1	S2
S3	X	X	S0

State	Encoding
S0	00
S1	01
S2	10
S3	11

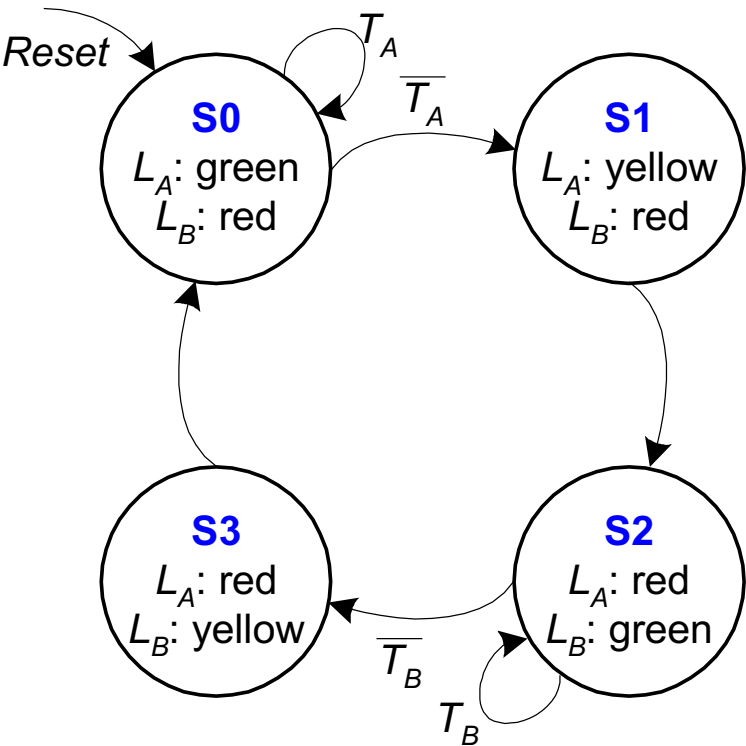
FSM State Transition Table



Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

FSM State Transition Table

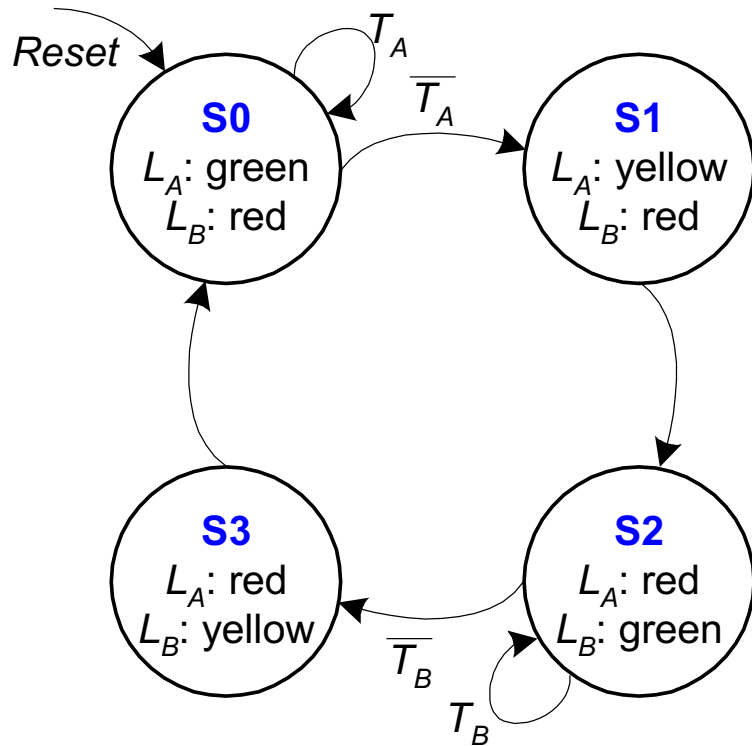


Current State		Inputs		Next State	
S ₁	S ₀	T _A	T _B	S' ₁	S' ₀
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

$S'_1 = ?$

State	Encoding
S0	00
S1	01
S2	10
S3	11

FSM State Transition Table

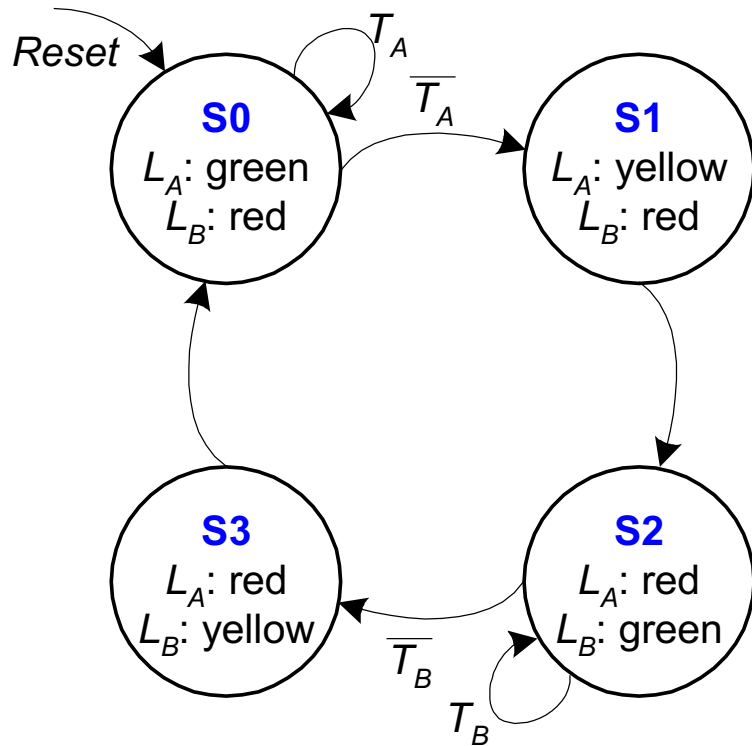


Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

$$S'_1 = (\overline{S_1} \cdot S_0) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B}) + (S_1 \cdot \overline{S_0} \cdot T_B)$$

FSM State Transition Table



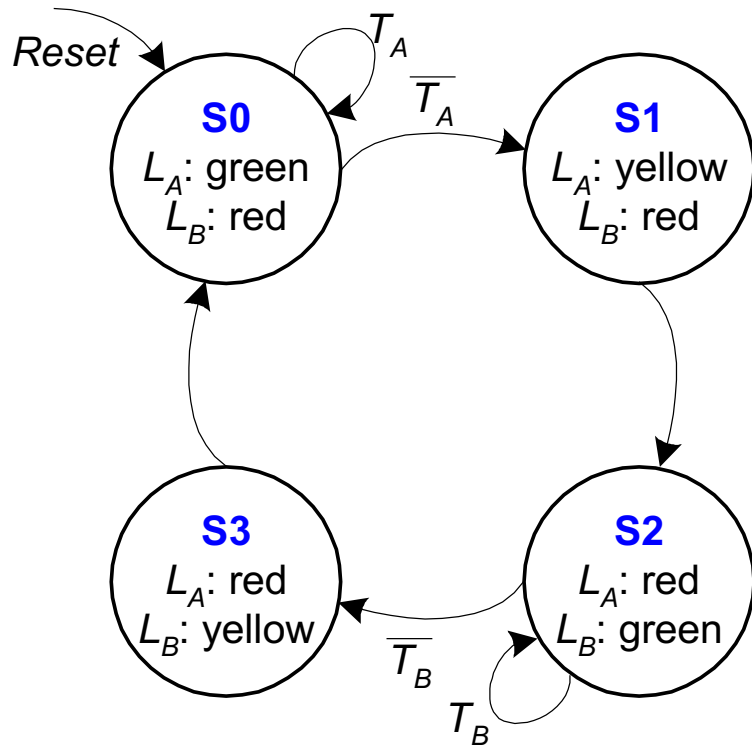
Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

$$S'_1 = (\overline{S_1} \cdot S_0) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B}) + (S_1 \cdot \overline{S_0} \cdot T_B)$$

$$S'_0 = ?$$

FSM State Transition Table



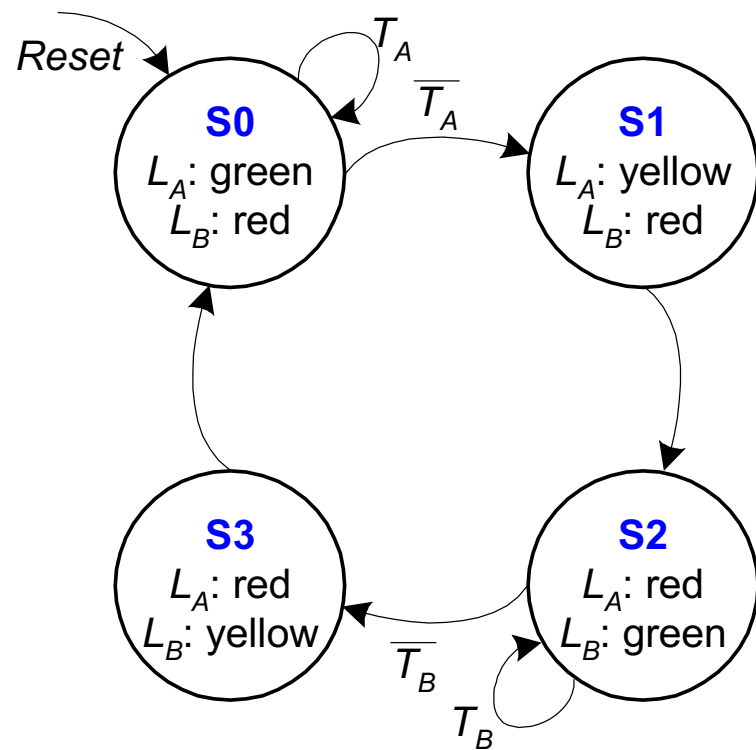
Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

$$S'_1 = (\overline{S_1} \cdot S_0) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B}) + (S_1 \cdot \overline{S_0} \cdot T_B)$$

$$S'_0 = (\overline{S_1} \cdot \overline{S_0} \cdot \overline{T_A}) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B})$$

FSM State Transition Table



Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

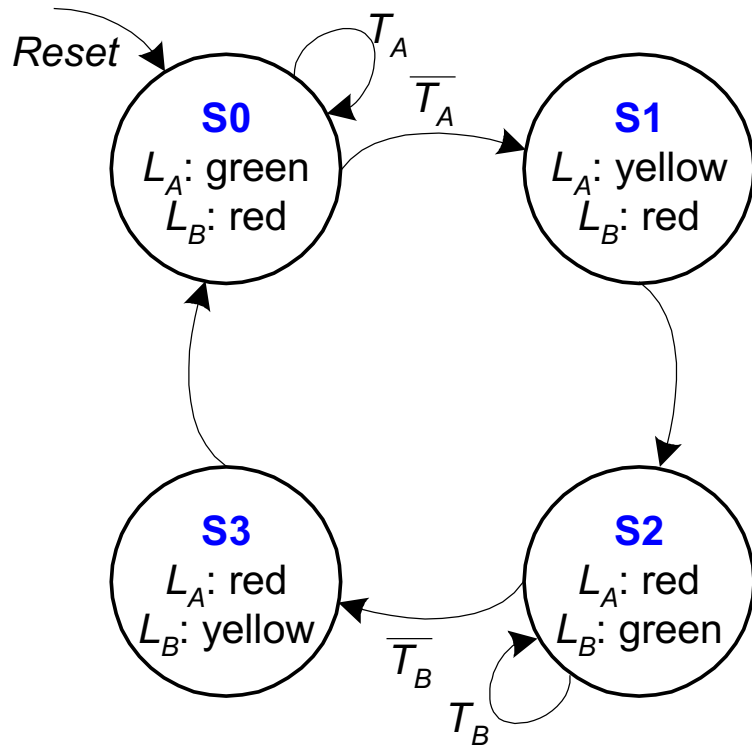
State	Encoding
S0	00
S1	01
S2	10
S3	11

$$S'_1 = S_1 \text{ xor } S_0 \quad \textbf{(Simplified)}$$

$$S'_0 = (\overline{S_1} \cdot \overline{S_0} \cdot \overline{T_A}) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B})$$

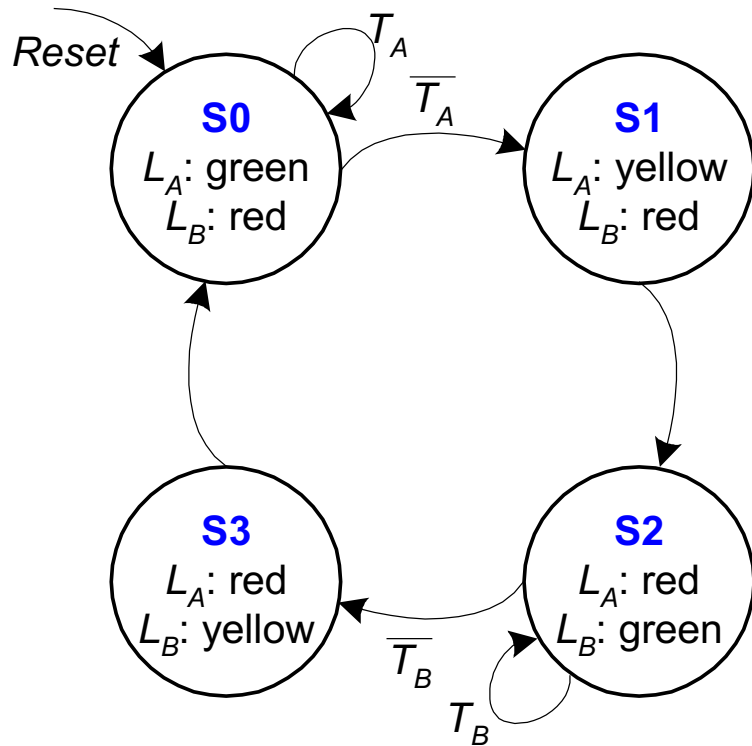
Finite State Machine: Output Table

FSM Output Table



Current State		Outputs	
S_1	S_0	L_A	L_B
0	0	green	red
0	1	yellow	red
1	0	red	green
1	1	red	yellow

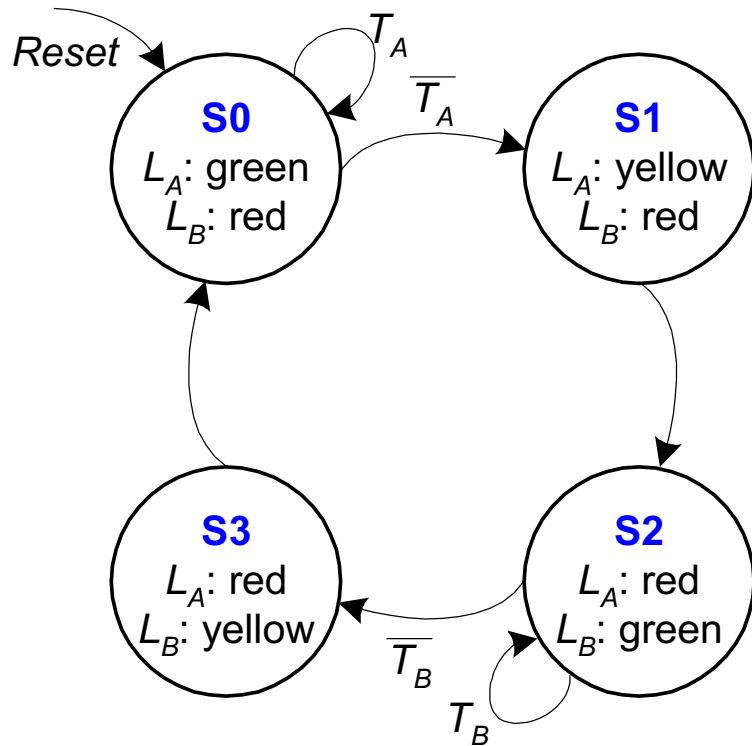
FSM Output Table



Current State		Outputs	
S_1	S_0	L_A	L_B
0	0	green	red
0	1	yellow	red
1	0	red	green
1	1	red	yellow

Output	Encoding
green	00
yellow	01
red	10

FSM Output Table

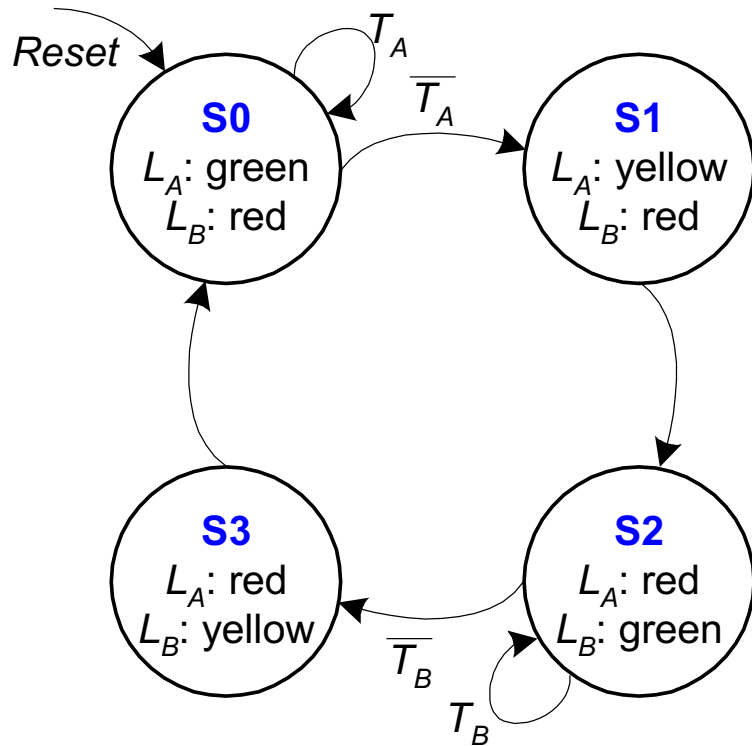


$$L_{A1} = S_1$$

Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

Output	Encoding
green	00
yellow	01
red	10

FSM Output Table



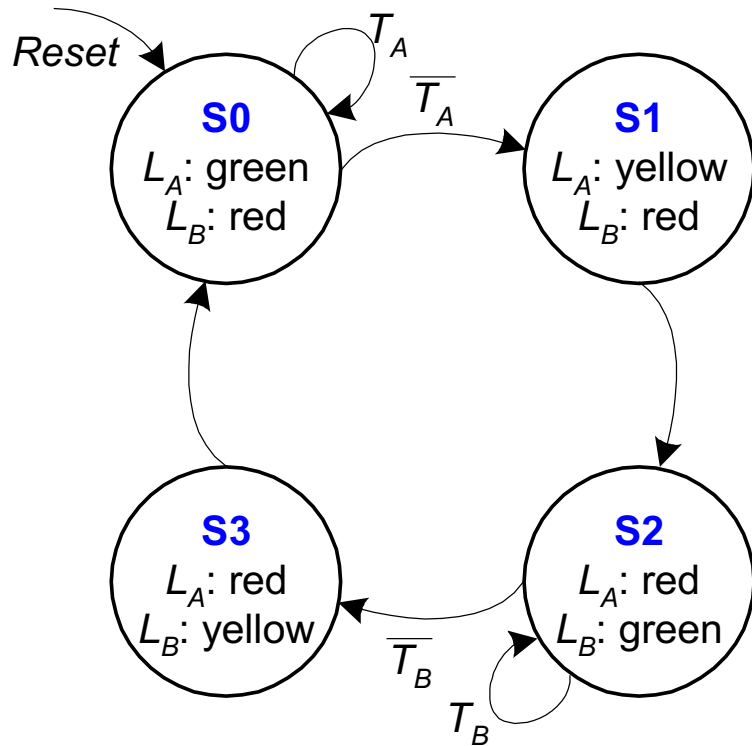
$$L_{A1} = S_1$$

$$L_{A0} = \overline{S_1} \cdot S_0$$

Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

Output	Encoding
green	00
yellow	01
red	10

FSM Output Table

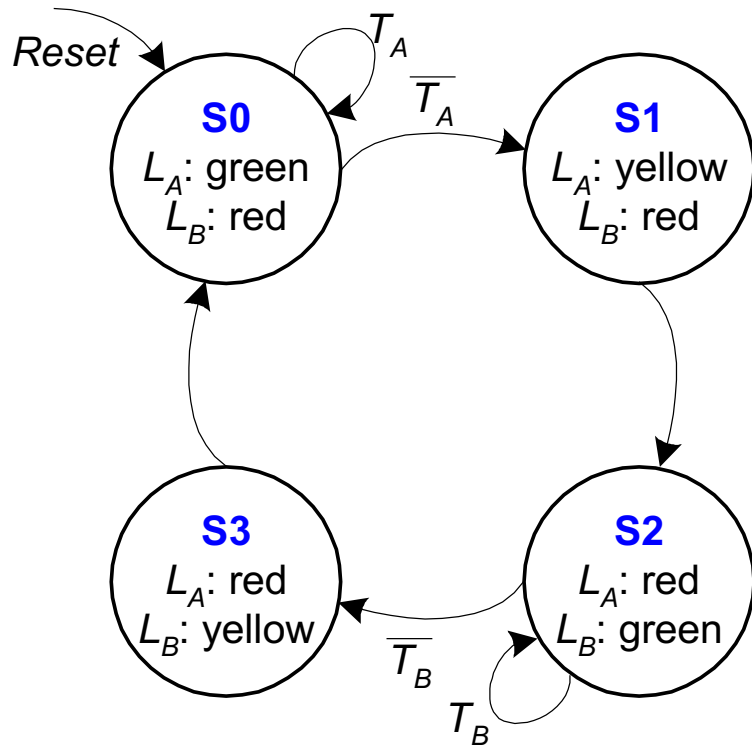


$$\begin{aligned}
 L_{A1} &= S_1 \\
 L_{A0} &= \overline{S_1} \cdot S_0 \\
 L_{B1} &= \overline{S_1}
 \end{aligned}$$

Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

Output	Encoding
green	00
yellow	01
red	10

FSM Output Table



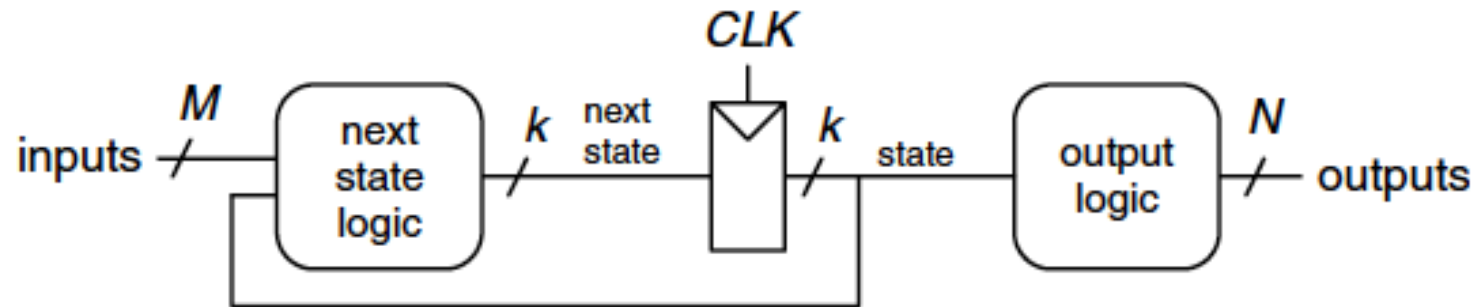
Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

$$\begin{aligned}
 L_{A1} &= S_1 \\
 L_{A0} &= \overline{S_1} \cdot S_0 \\
 L_{B1} &= \overline{S_1} \\
 L_{B0} &= S_1 \cdot S_0
 \end{aligned}$$

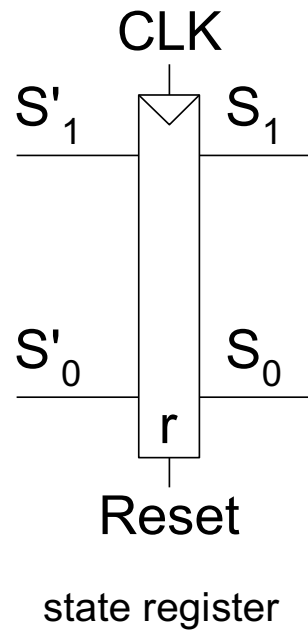
Output	Encoding
green	00
yellow	01
red	10

Finite State Machine: Schematic

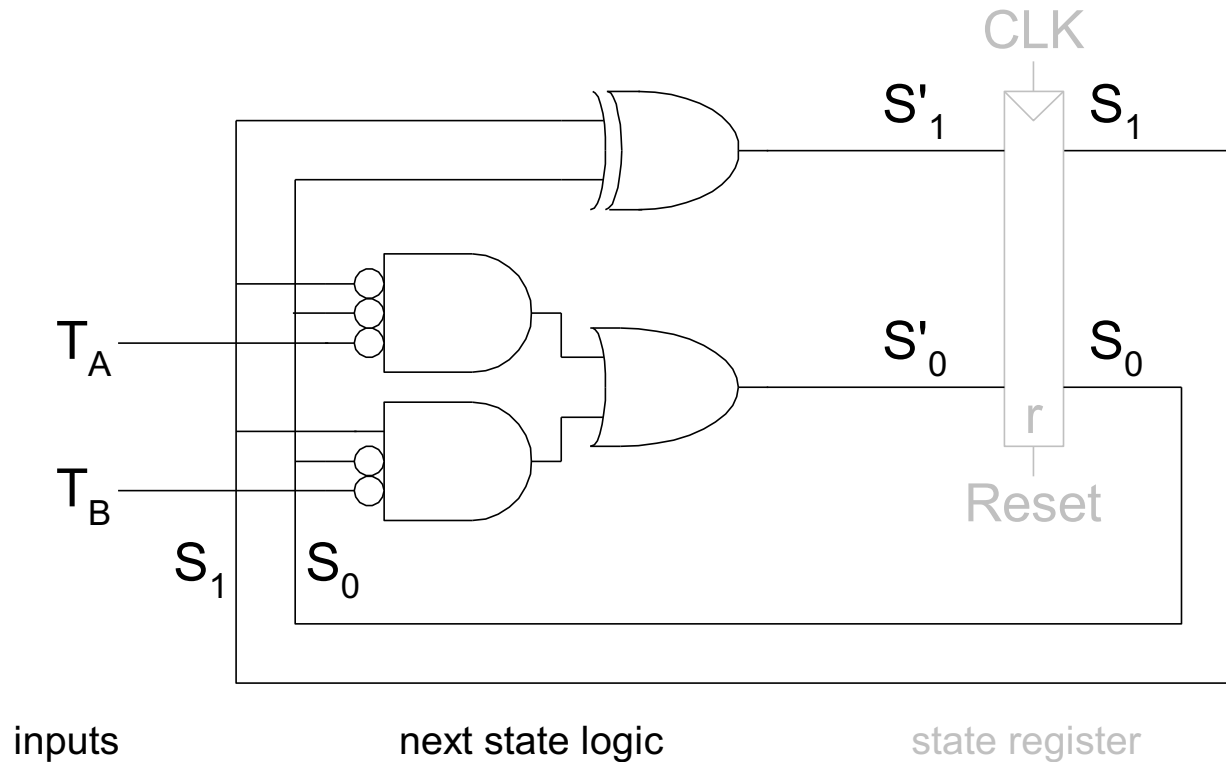
FSM Schematic: State Register



FSM Schematic: State Register



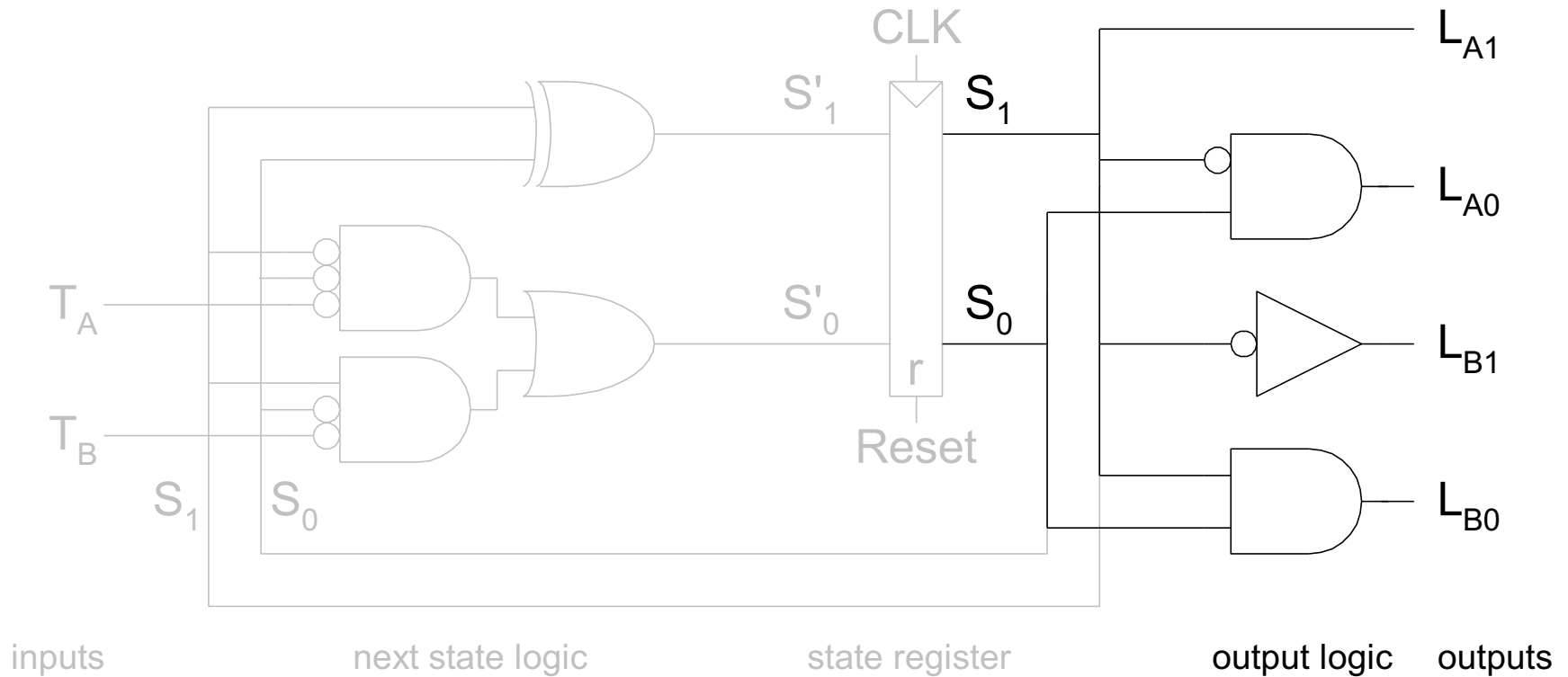
FSM Schematic: Next State Logic



$$S'_1 = S_1 \text{ xor } S_0$$

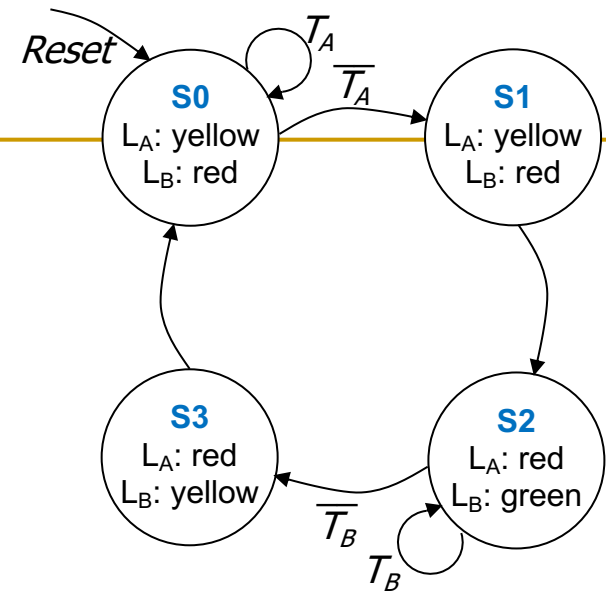
$$S'_0 = (\overline{S_1} \cdot \overline{S_0} \cdot \overline{T_A}) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B})$$

FSM Schematic: Output Logic



$$\begin{aligned}L_{A1} &= S_1 \\L_{A0} &= \overline{S_1} \cdot S_0 \\L_{B1} &= \overline{S_1} \\L_{B0} &= S_1 \cdot S_0\end{aligned}$$

FSM Timing Diagram



CLK_

Reset_

T_A _

T_B _

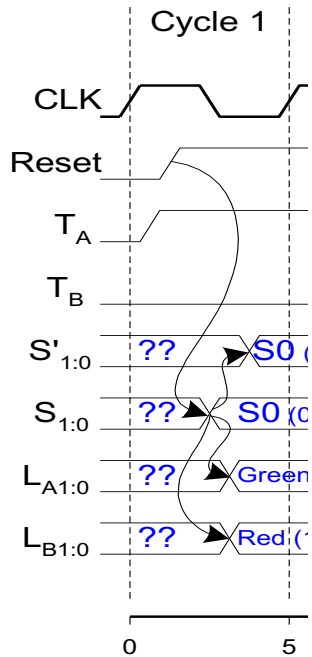
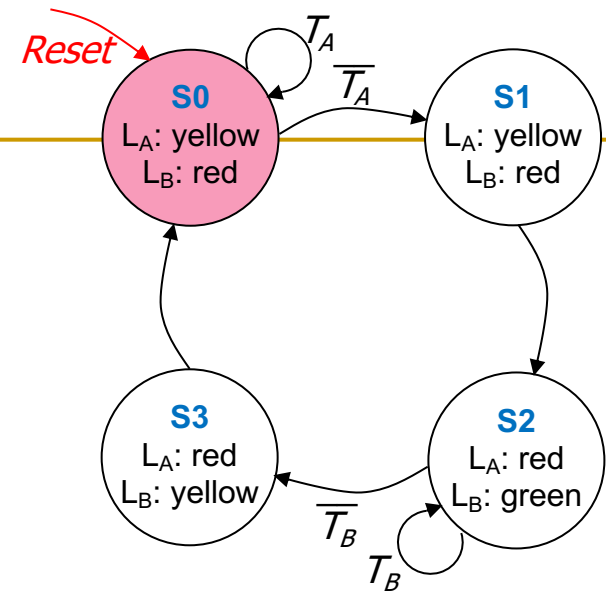
S'_{1:0} _

S_{1:0} _

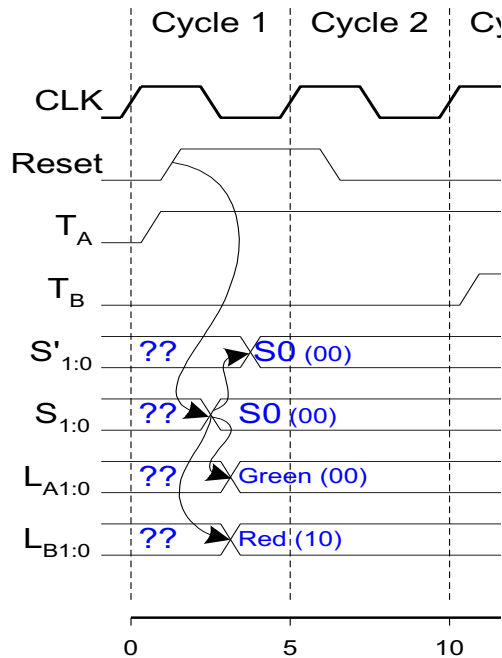
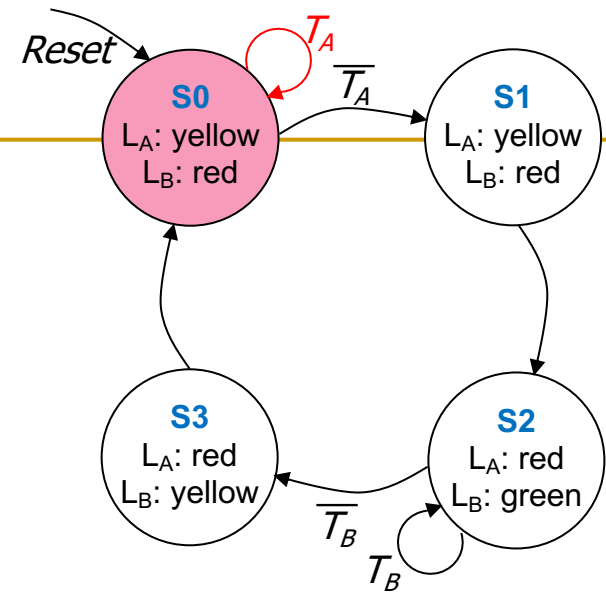
L_{A1:0} _

L_{B1:0} _

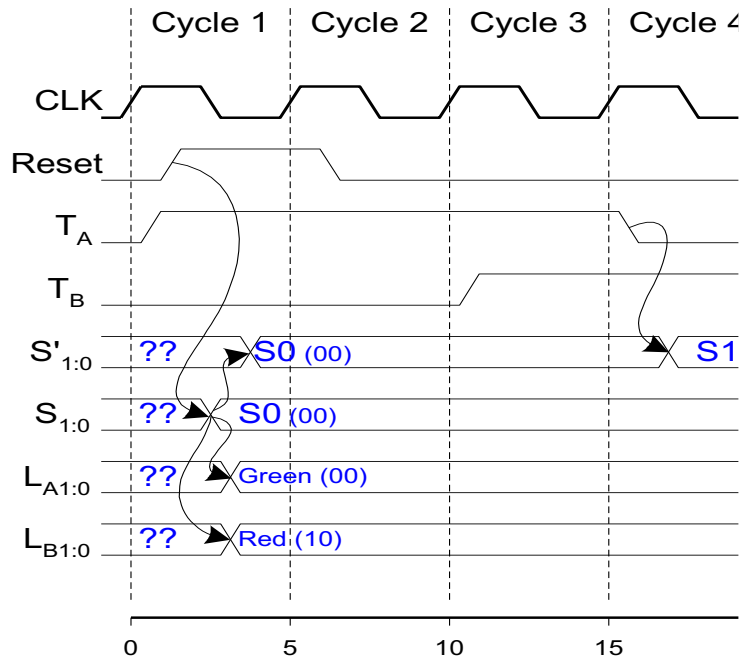
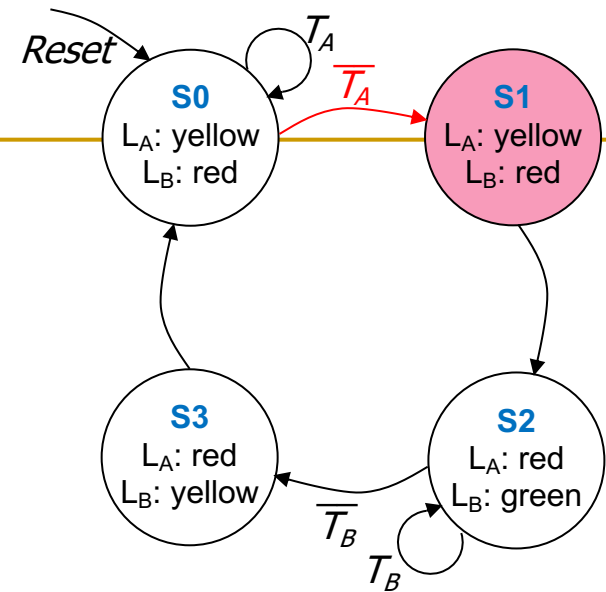
FSM Timing Diagram



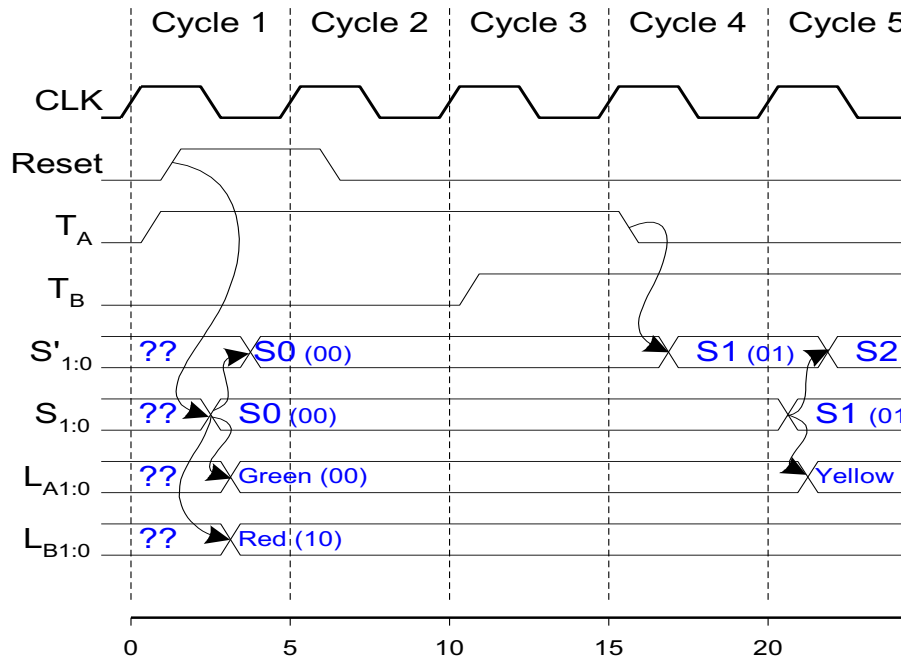
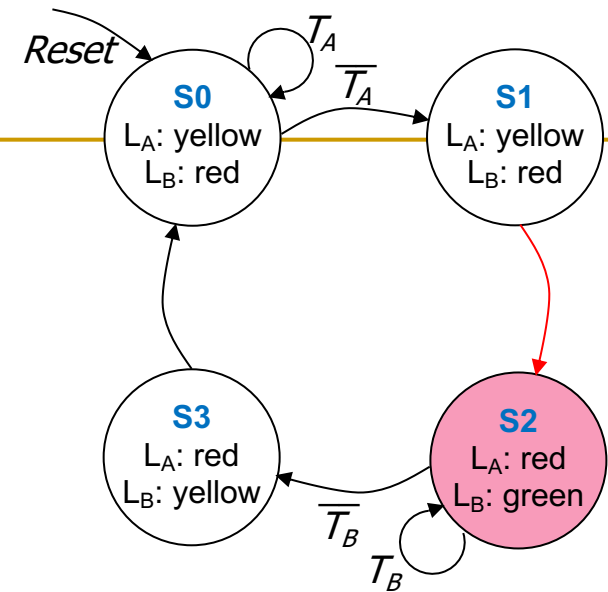
FSM Timing Diagram



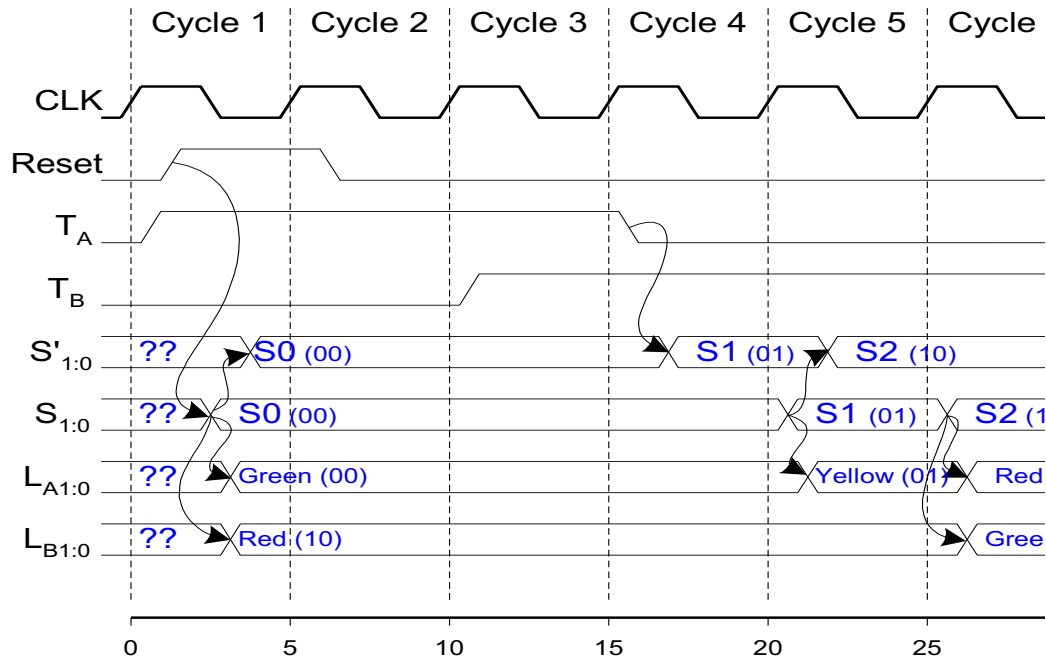
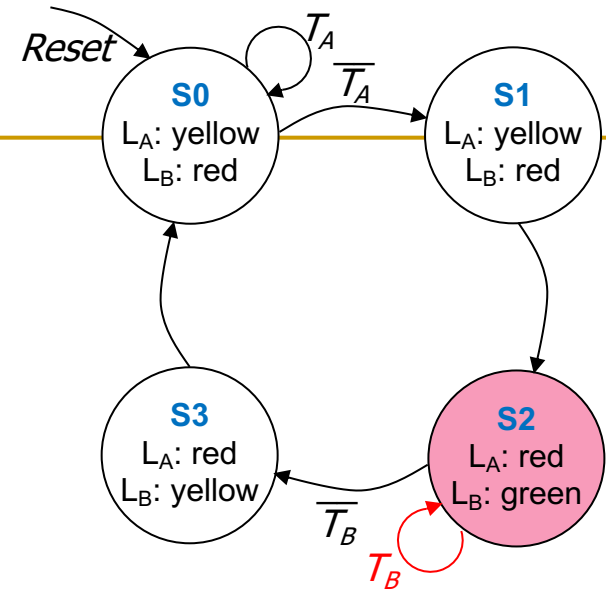
FSM Timing Diagram



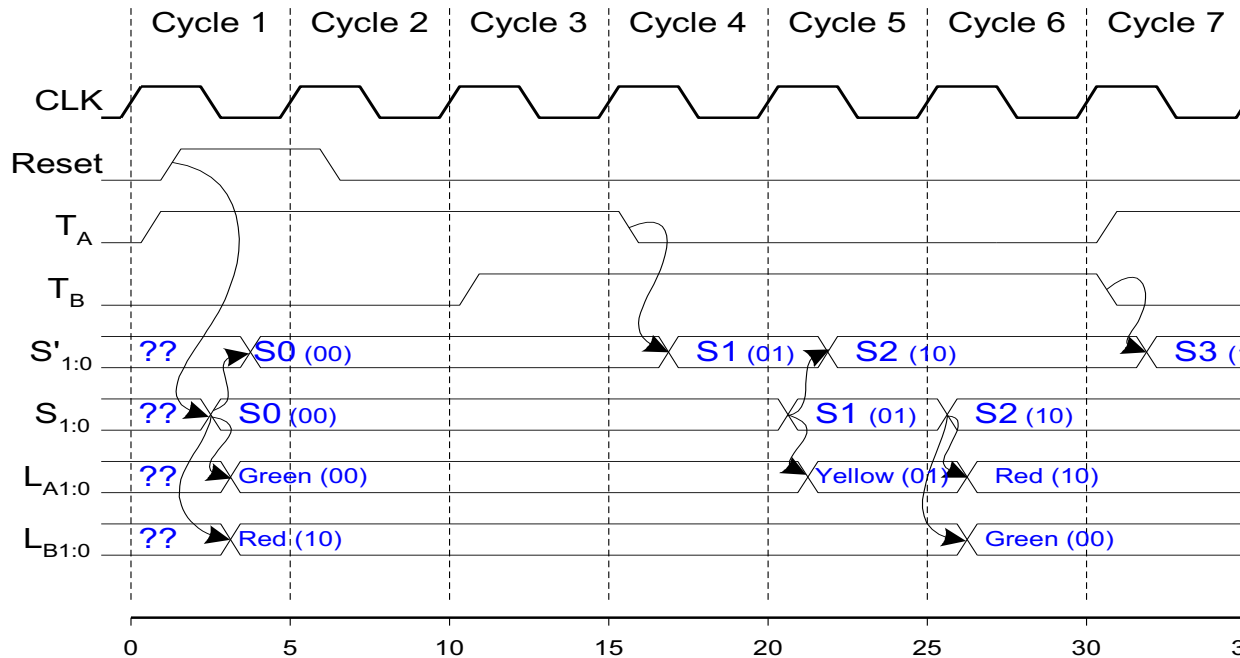
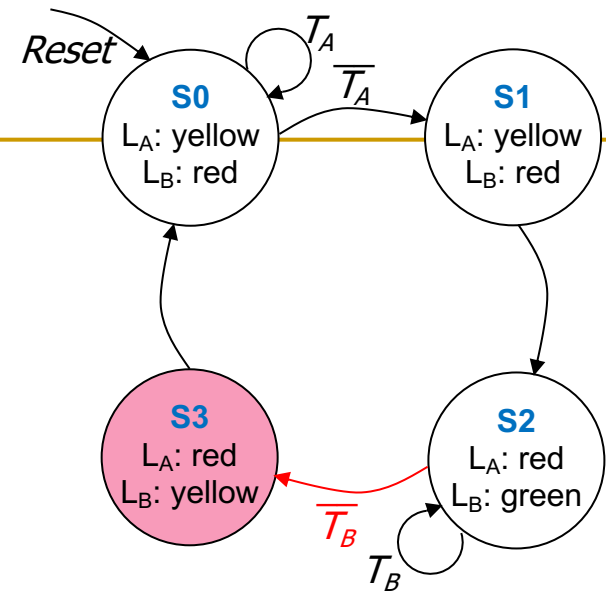
FSM Timing Diagram



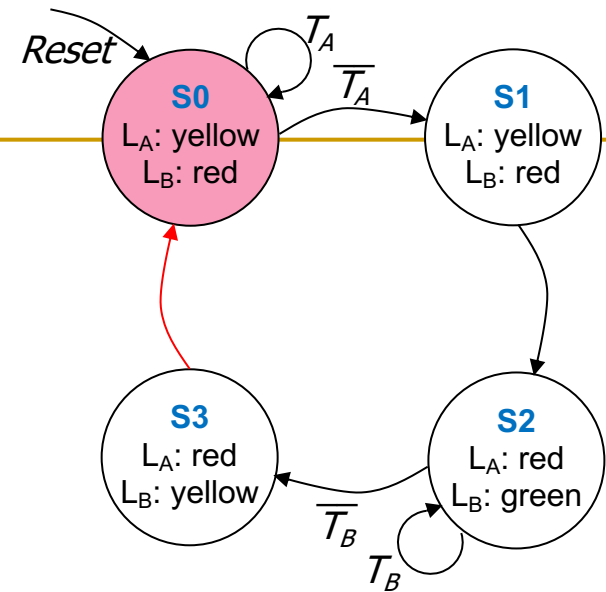
FSM Timing Diagram



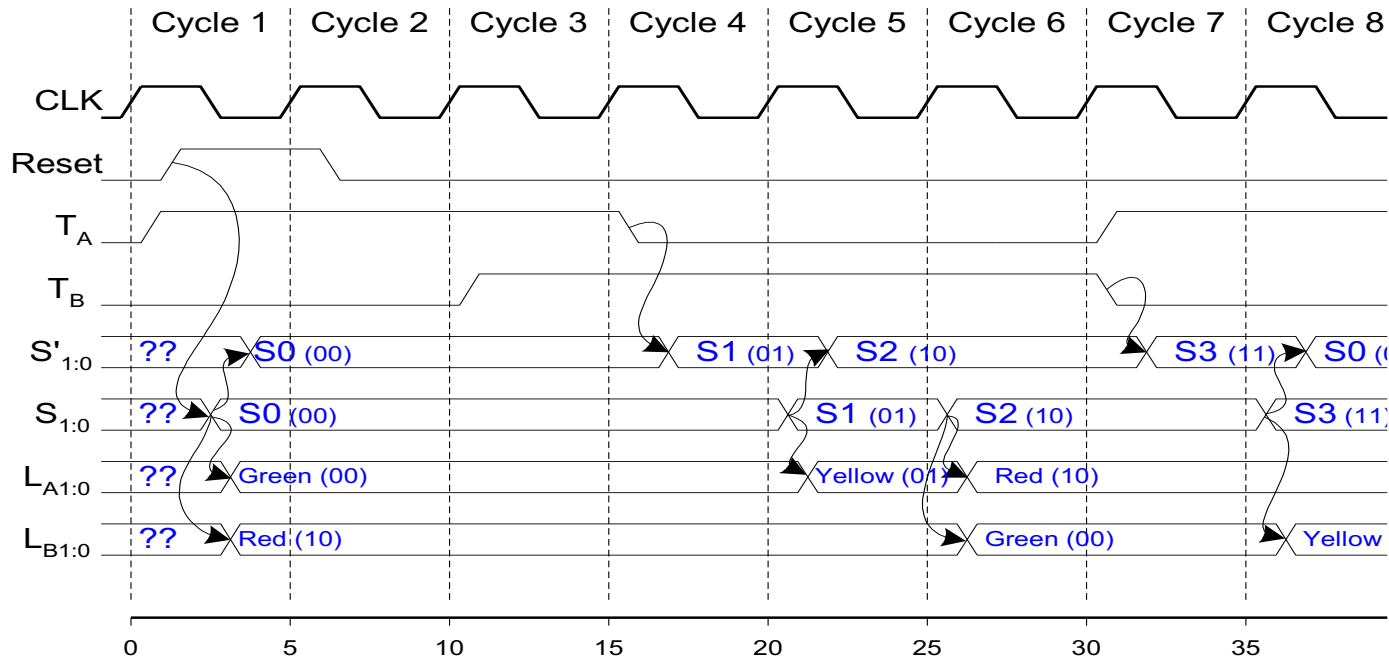
FSM Timing Diagram



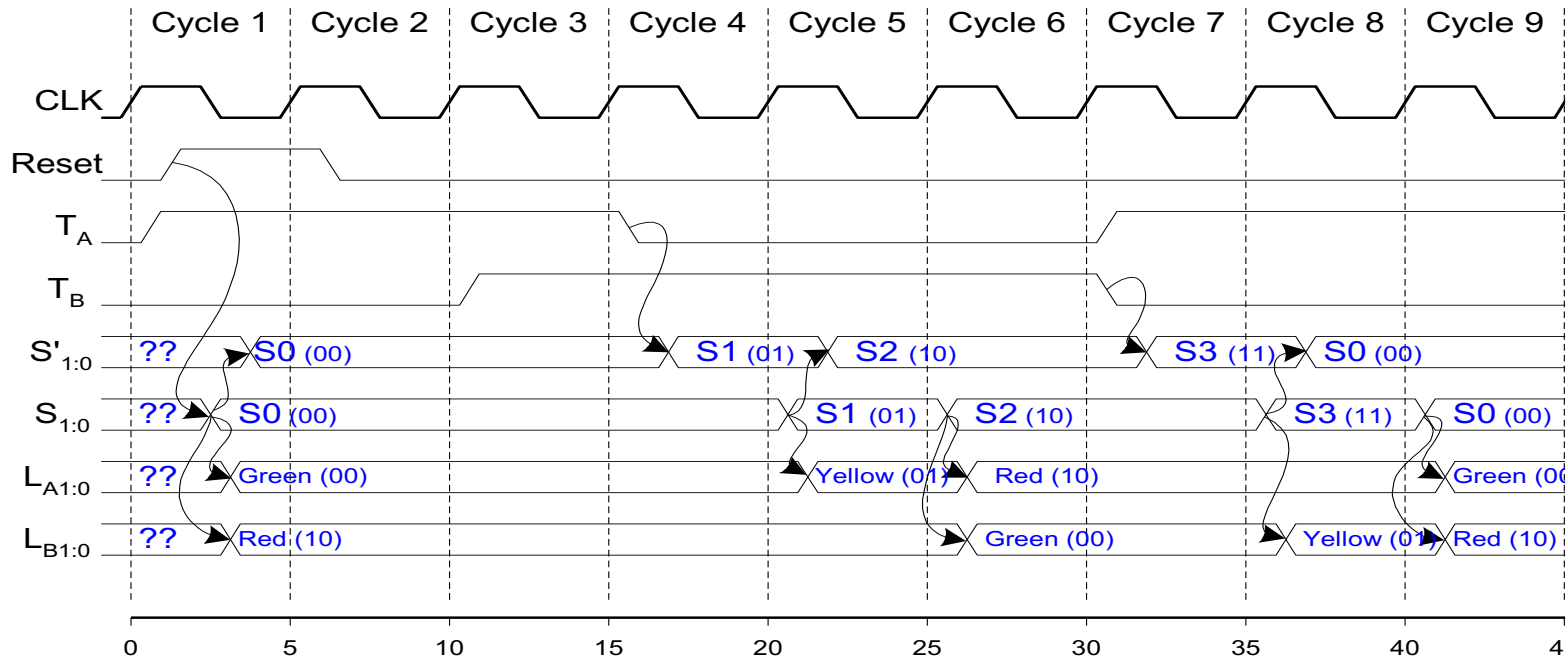
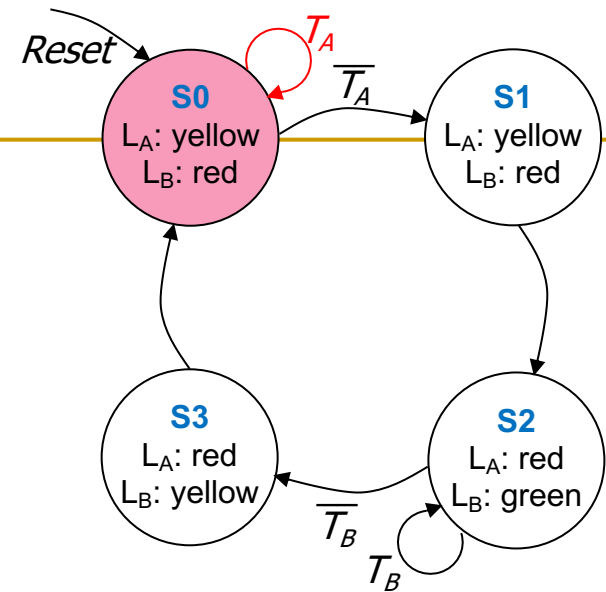
FSM Timing Diagram



This is from H&H Section 3.4.1

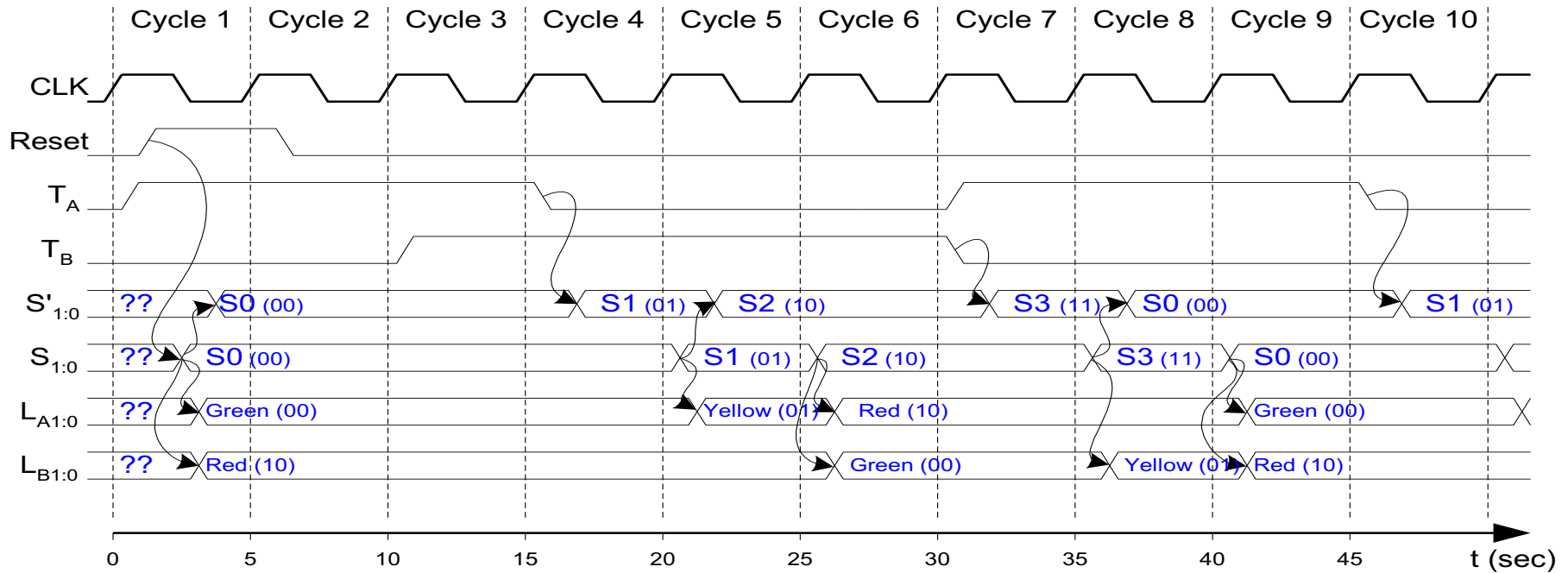
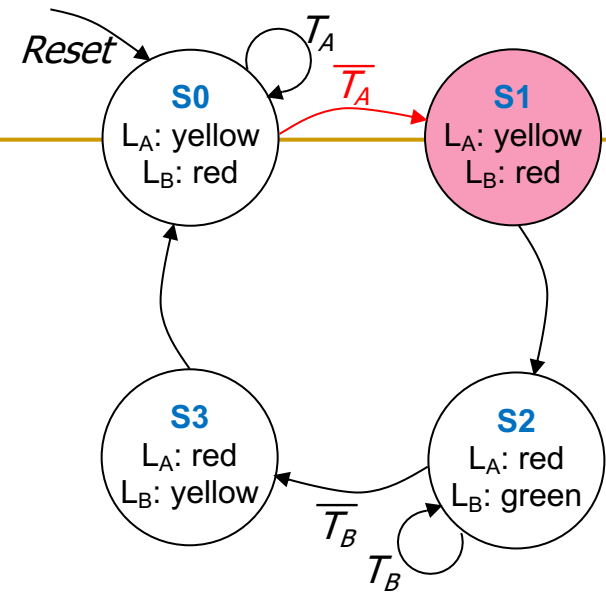


FSM Timing Diagram



FSM Timing Diagram

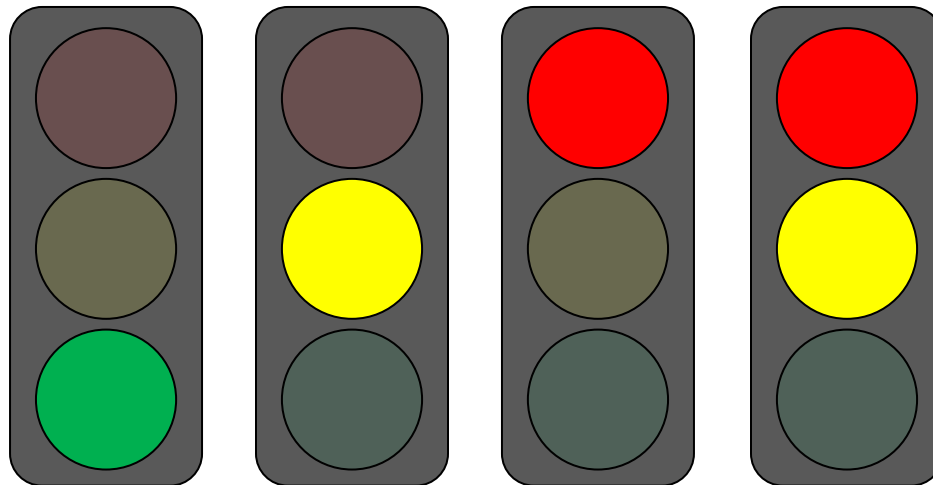
See H&H Chapter 3.4



Finite State Machine: State Encoding

FSM State Encoding

- How do we encode the state bits?
 - Three common state binary encodings with different tradeoffs
 1. **Fully Encoded**
 2. **1-Hot Encoded**
 3. **Output Encoded**
- Let's see an example **Swiss** traffic light with 4 states
 - Green, Yellow, Red, Yellow+Red



FSM State Encoding (II)

1. Binary Encoding (Full Encoding):

- ❑ Use the minimum possible number of bits
 - Use $\log_2(num_states)$ bits to represent the states
- ❑ *Example state encodings: 00, 01, 10, 11*
- ❑ **Minimizes** # flip-flops, but not necessarily output logic or next state logic

2. One-Hot Encoding:

- ❑ Each bit encodes a different state
 - Uses num_states bits to represent the states
 - Exactly 1 bit is “hot” for a given state
- ❑ *Example state encodings: 0001, 0010, 0100, 1000*
- ❑ **Simplest design process** – very automatable
- ❑ **Minimizes** next state logic, **maximizes** # flip-flops

FSM State Encoding (III)

3. Output Encoding:

- ❑ Outputs are **directly accessible** in the state encoding
- ❑ For example, since we have **3 outputs** (light color), encode state with **3 bits**, where each bit represents a color
- ❑ *Example states:* 001, 010, 100, 110
 - Bit₀ encodes **green** light output,
 - Bit₁ encodes **yellow** light output
 - Bit₂ encodes **red** light output
- ❑ **Minimizes** output logic
- ❑ Only works for Moore Machines (output function of state)

FSM State Encoding (III)

3. Output Encoding:

- ❑ Outputs are **directly accessible** in the state encoding

The **designer** must **carefully** choose an encoding scheme to **optimize** the design under given constraints

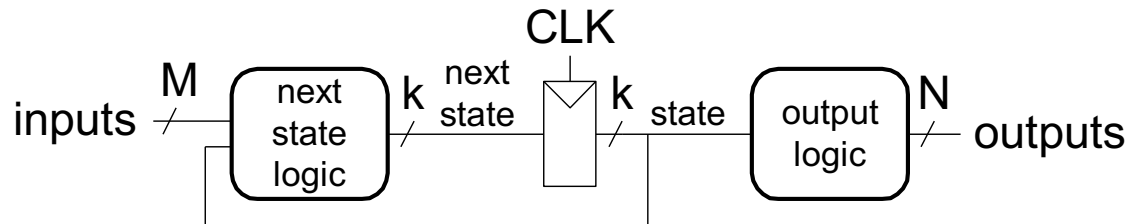
- ❑ **Minimizes** output logic
- ❑ Only works for Moore Machines (output depends only on state)

Moore vs. Mealy Machines

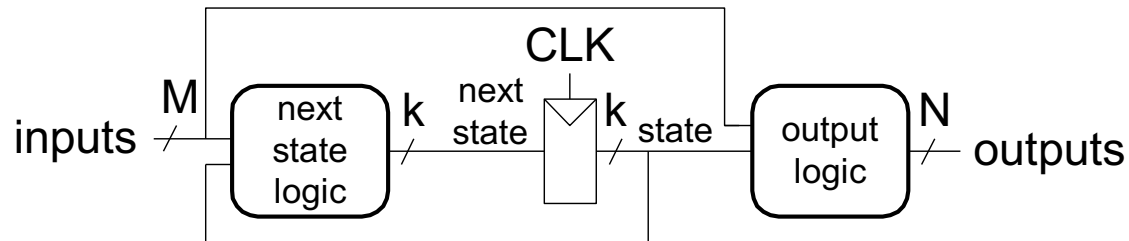
Recall: Moore vs. Mealy FSMs

- Next state is determined by the current state and the inputs
- Two types of FSMs differ in the **output logic**:
 - **Moore FSM**: outputs depend only on the current state
 - **Mealy FSM**: outputs depend on the current state and the inputs

Moore FSM



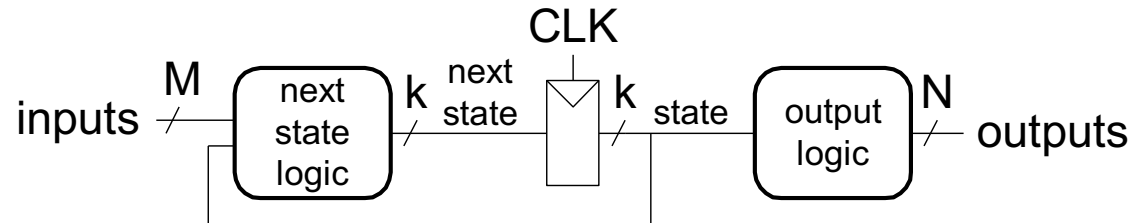
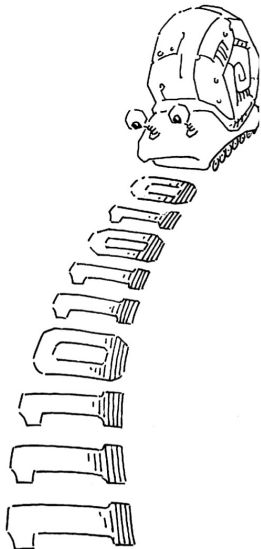
Mealy FSM



Moore vs. Mealy FSM Examples

- Alyssa P. Hacker has a snail that crawls down a paper tape with 1's and 0's on it.
- The snail smiles whenever the last four digits it has crawled over are **1101**.
- Design Moore and Mealy FSMs of the snail's brain.

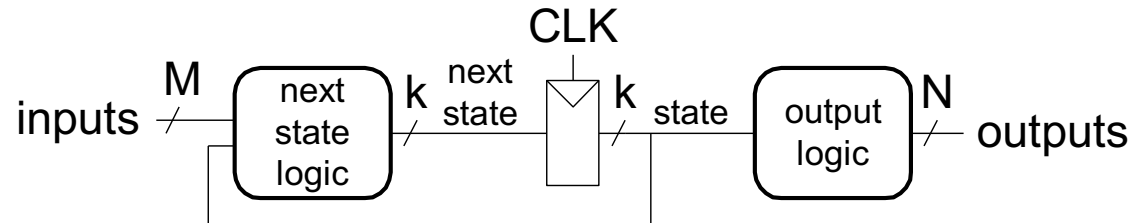
Moore FSM



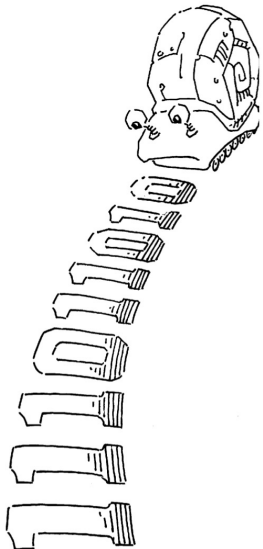
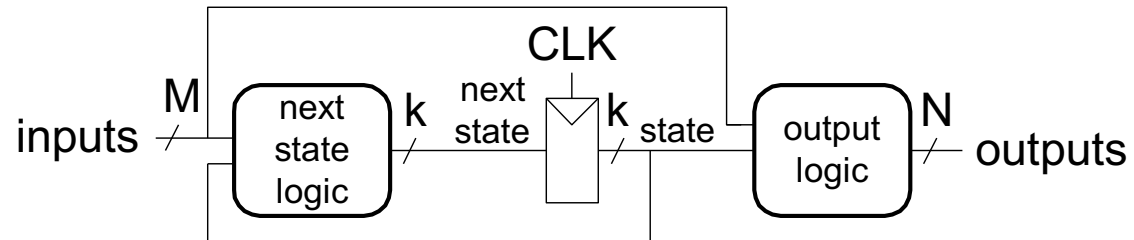
Moore vs. Mealy FSM Examples

- Alyssa P. Hacker has a snail that crawls down a paper tape with 1's and 0's on it.
- The snail smiles whenever the last four digits it has crawled over are **1101**.
- Design Moore and Mealy FSMs of the snail's brain.

Moore FSM

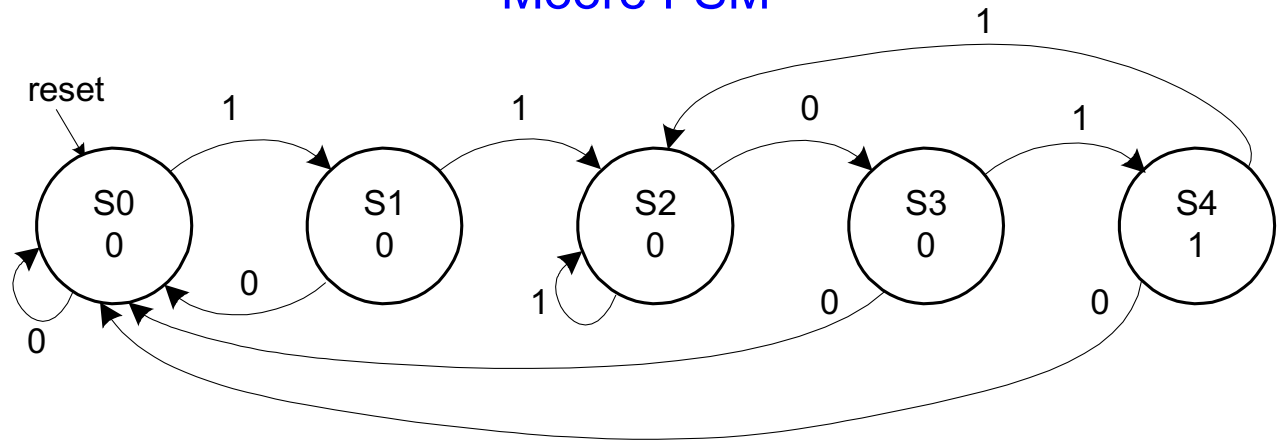


Mealy FSM



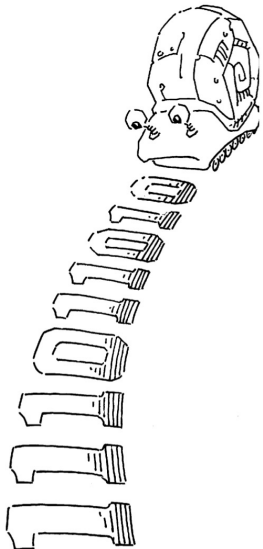
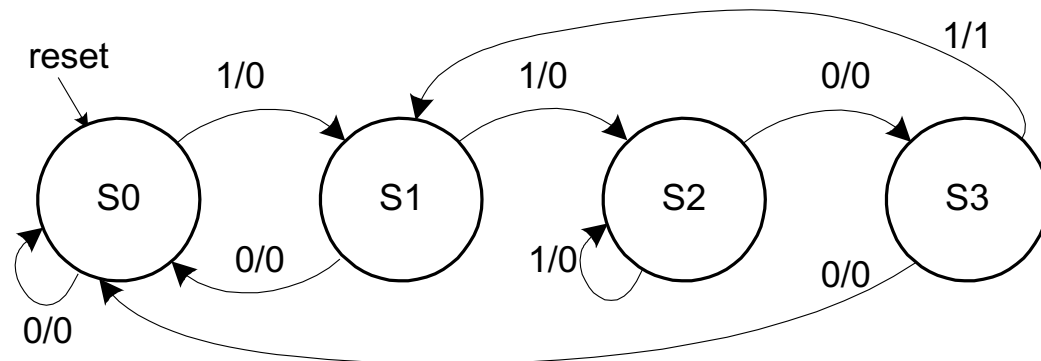
State Transition Diagrams

Moore FSM



What are the tradeoffs?

Mealy FSM



FSM Design Procedure

- **Determine** all possible states of your machine
- **Develop** a **state transition diagram**
 - Generally this is done from a textual description
 - You need to 1) determine the **inputs** and **outputs** for each **state** and 2) figure out how to get from one state to another
- **Approach**
 - Start by defining the **reset state** and what happens from it – this is typically an easy point to start from
 - Then continue to add **transitions** and **states**
 - Picking **good state names** is very important
 - Building an FSM is **like** programming (but it *is not* programming!)
 - An FSM has a sequential “control-flow” like a program with conditionals and goto’s
 - The if-then-else construct is controlled by one or more inputs
 - The outputs are controlled by the state or the inputs
 - In hardware, we typically have many concurrent FSMs

What is to Come: LC-3 Processor

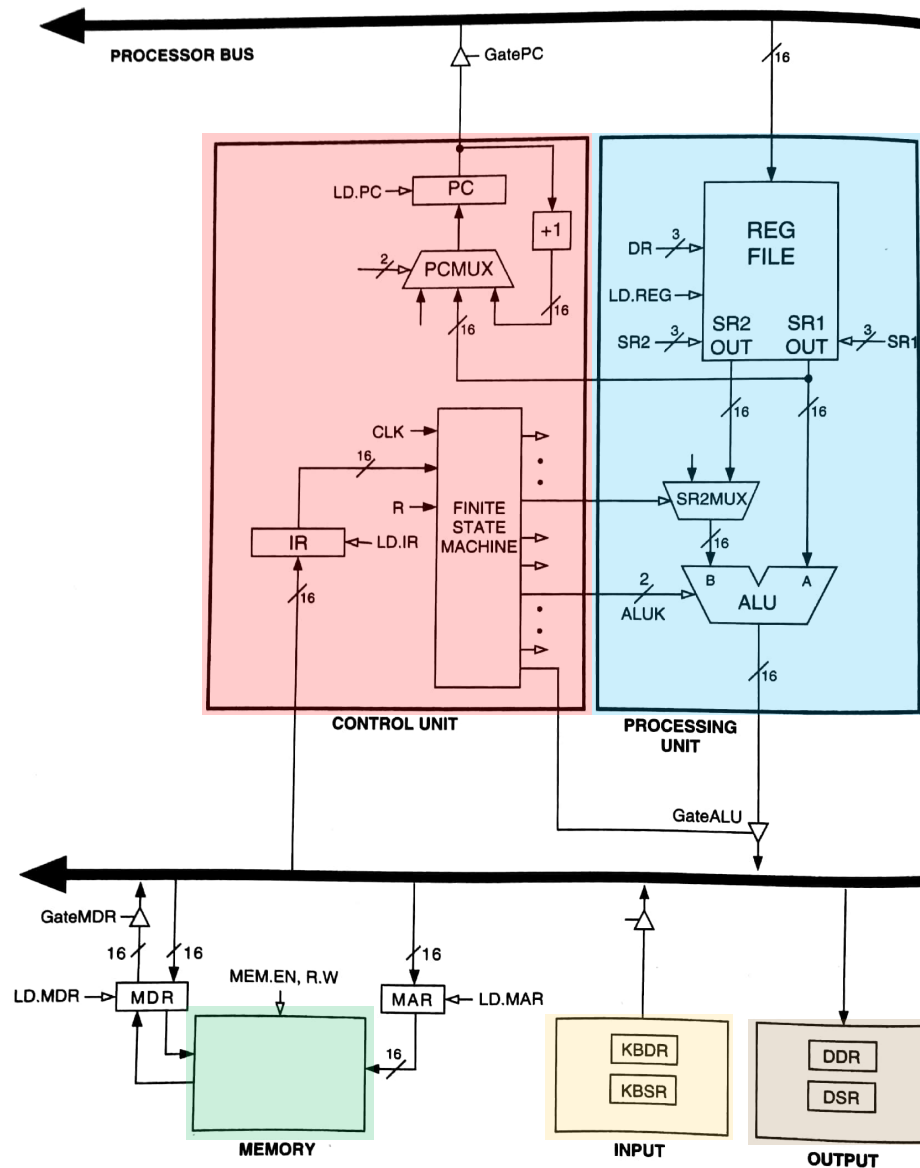
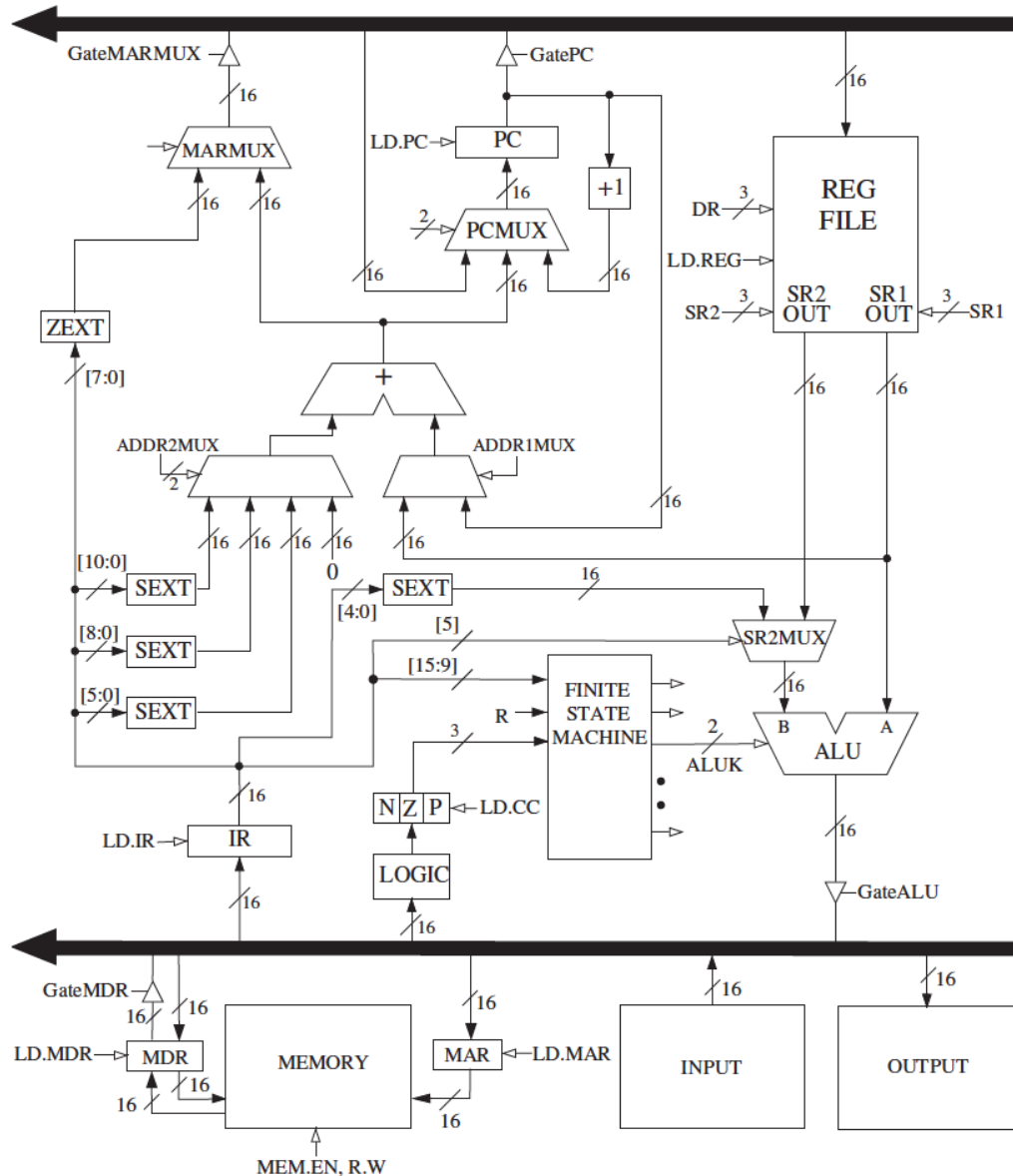
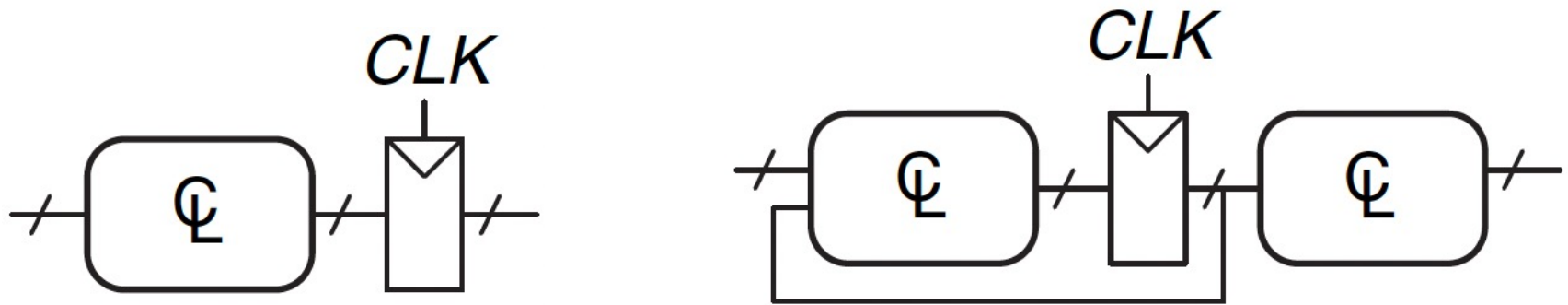


Figure 4.3 The LC-3 as an example of the von Neumann model

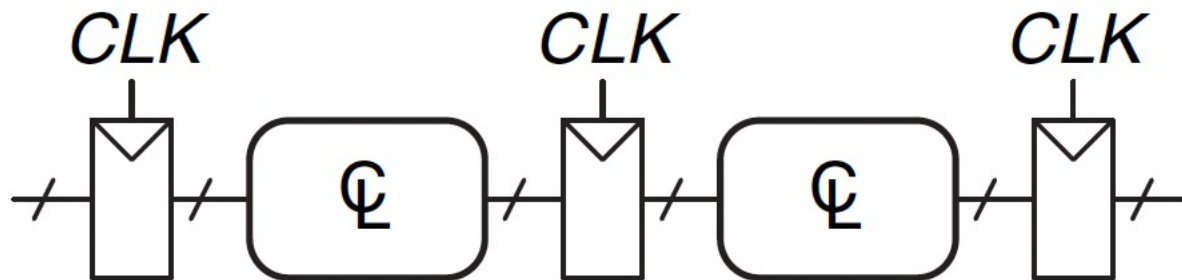
What is to Come: LC-3 Datapath



Recall: Synchronous Sequential Circuits



Major types of Sequential Logic Circuits



Digital Design & Computer Arch.

Lecture 4a: Sequential Logic Design II Finite State Machines

Prof. Onur Mutlu

ETH Zürich

Spring 2026

27 February 2026